

오픈소스소프트웨어 팀 프로젝트 안내서

2026학년도 2학기

국립부경대학교 컴퓨터·인공지능공학부

오상호 교수 (shoh0320@pknu.ac.kr)

문서 버전 v1.3 · 2026년 9월

목차

안내서 사용법.....	5
1. 프로젝트 개요.....	7
1.1 한 줄 요약.....	7
1.2 프로젝트 목적.....	7
1.3 프로젝트 완성 기준 5개.....	7
1.4 제출 일정 한눈에 보기 (마감은 모두 수업일 전날 목요일 18:00).....	8
2. 팀 구성과 역할.....	9
2.1 팀 구성 규칙.....	9
2.2 역할 4개.....	9
2.3 팀 약속 (2주차, 부록 F 양식).....	9
2.4 팀에 문제가 생겼을 때.....	9
3. 15주 로드맵.....	10
3.1 단계별 목표.....	11
3.2 주차별 상세.....	11
4. 주제 정하기 (2~3주차).....	13
4.1 좋은 주제 고르는 질문 3개.....	13
4.2 규모.....	13
4.3 유형별 예시.....	13
4.4 예시: 가계부 CLI 'moneylog' — 기능 나누기.....	13
4.5 피해야 할 주제.....	14
5. 프로젝트 제안서 (5주차 10/2 발표).....	15
5.1 항목 7개.....	15
5.2 제출과 발표.....	15
6. 준비하기: 계정과 도구 (3~6주차, 각자).....	16
6.1 필요한 것.....	16
6.2 GitHub 계정과 토큰 (수업 3주차).....	16
6.3 Git 설치와 첫 설정 (수업 3주차).....	16
6.4 소스트리와 VS Code (수업 6주차).....	17
6.5 Python 설치.....	17
6.6 준비 완료 체크리스트 (6주차까지).....	18
7. 팀 저장소 만들기 (2~6주차).....	19
7.1 저장소 생성 (팀장, 2주차).....	19
7.2 팀원 협력자(Collaborator) 등록 (수업 11주차 '협력자 등록'과 같은 화면).....	19
7.3 기본 파일 넣기 (팀장, 5주차 이후, GitHub 웹에서).....	19
7.4 소스트리로 클론 (전원, 6주차).....	20
7.5 main 브랜치 보호 (팀장, 9주차) — 수업 13주차 '브랜치 보호하기'와 동일.....	20

7.6 선택: 라벨과 Project 보드.....	21
8. 협업 루틴 — 매주 반복하는 8단계	22
8.1 브랜치 규칙 (수업 7주차 '실습 브랜치 규칙'과 동일).....	22
8.2 8단계 루틴 (소스트리 기준).....	22
8.3 커밋 메시지	23
8.4 이슈	23
8.5 풀 리퀘스트 (수업 9·13주차)	23
8.6 리뷰하는 법 (수업 7주차 '사람들이 중요하다', 9주차 '검토·개정·협업')	24
8.7 충돌(Conflict) 해결 — 수업 9주차 '충돌 병합 해결하기'와 동일	24
8.8 자주 하는 실수와 복구 (수업 12주차 '실무 사례와 함께 Git 다루기' 기능 활용)	25
9. Python 프로젝트 기본	26
9.1 Python 버전	26
9.2 폴더 구조	26
9.3 설치와 실행 (README에 그대로 적을 내용).....	26
9.4 코드 습관 (수업 3주차 '스타일 가이드').....	27
9.5 저장소에 올리면 안 되는 것.....	27
10. 활동 규칙 (평가 30점)	28
10.1 규칙 표	28
10.2 매주 스스로 확인하는 방법.....	28
10.3 항목별 정의·예시·목적.....	29
11. 타 팀 저장소에 기여하기 (11~13주차)	30
11.1 개요와 매칭	30
11.2 받는 팀의 준비 (11주차 수업 전까지, 문서 담당)	30
11.3 기여하는 절차 (팀 단위, 수업 11주차 그대로)	30
11.4 외부 오픈소스 프로젝트 기여 (선택).....	31
12. 회의록 (총 10회)	32
12.1 제출 날짜	32
12.2 회의록 내용 (부록 B 양식)	32
13. 프로젝트 중간발표 (10주차 11/6)	33
13.1 중간보고서	33
13.2 중간발표	33
14. 마무리: 릴리스·최종보고서·최종발표 (13~15주차)	34
14.1 v1.0.0 태그와 릴리스 (14주차, 수업 9주차 '프로그램 출시하기').....	34
14.2 README 필수 항목 7개 (수업 13주차 '프로젝트에 꼭 포함할 파일').....	34
14.3 최종보고서	34
14.4 최종발표 (15주차 12/18)	35
15. 평가.....	36
15.1 제안서 10점 + 발표 5점.....	36
15.2 중간보고서 5점 + 중간발표 5점.....	36

15.3 최종보고서 10점 + 최종발표 5점	37
15.4 활동 규칙 30점.....	37
15.5 결과물 30점 (완성 기준 5개).....	37
15.6 감점·무효.....	37
16. 도움 받기.....	38
16.1 막혔을 때 순서	38
16.2 질문 양식	38
16.3 AI 도구 (Copilot, ChatGPT, Claude 등).....	38
16.4 참고 자료	38
부록 A. 주차별 개인 점검표.....	39
부록 B. 회의록 양식.....	40
부록 C. 템플릿 파일.....	41
C-1. README.md.....	41
C-2. CONTRIBUTING.md	42
C-3. CODE_OF_CONDUCT.md (행동 수칙).....	42
C-4. requirements.txt	43
C-5. main.py (시작 파일 예시)	43
C-6. .gitignore에 추가할 줄 (저장소 생성 시 고른 Python 템플릿 아래에)	44
C-7. 풀 리퀘스트 설명 양식 (선택: .github/PULL_REQUEST_TEMPLATE.md 로 저장하면 자동 입력) ..	44
부록 D. 소스트리·명령어 대응표와 충돌 실습.....	45
D-1. 소스트리 조작 ↔ Git Bash 명령 (수업 3·12·14주차).....	45
D-2. 충돌 해결 실습 (9주차, 수업 9주차 직후, 2인 1조 20분)	45
부록 E. 문제 해결 FAQ	47
부록 F. 팀 약속 양식.....	48

안내서 사용법

이 안내서는 Git과 GitHub를 처음 접하는 학생을 기준으로 썼습니다. 사용하는 도구와 방법은 모두 수업 강의자료와 같습니다. 수업에서 배운 것을 그 주에 팀 저장소에 적용하면 프로젝트가 진행되도록 짜여 있으니, 매주 '이번 주 할 일'만 따라오면 됩니다.

구성

부	장	내용	읽는 시기
1부 개요	1~3장	목적, 완성 기준, 팀 구성, 15주 로드맵	지금 바로
2부 계획	4~5장	주제 정하기, 제안서	2~5주차
3부 준비	6~7장	계정·도구 준비, 팀 저장소 만들기	3~6주차
4부 협업	8~10장	매주 반복하는 협업 루틴, Python 프로젝트 기본, 활동 규칙(평가 30점)	6주차부터 매주
5부 확장·마무리	11~16장	타 팀 기여, 회의록, 중간발표, 릴리스·최종보고서·발표, 평가, 도움 받기	10주차부터
부록	A~F	점검표, 회의록 양식, 템플릿 파일, 소스트리·명령어 대응표와 충돌 실습, FAQ, 팀 약속	필요할 때

표기 규칙

- 이렇게 표시된 글자는 화면의 버튼·메뉴 이름, 파일 이름, 또는 Git Bash 에 입력하는 명령
- [필수]는 평가에 반영되는 항목, [주의]는 자주 하는 실수, [TIP]은 요령, [수업]은 같은 내용을 배우는 수업 주차
- '각자'는 4명 모두에게 적용되는 기준, '팀'은 팀 단위로 1 회면 되는 기준
- '주차'는 수업 회차입니다(예: 5주차 = 10/2 수업). 모든 제출물의 제출 마감은 **수업일 전날(목요일) 18:00**
- 소스트리(GUI)와 Git Bash(명령어) 중 편한 쪽을 쓰면 됩니다. 본문은 수업 6~13 주차에 쓰는 소스트리를 기준으로 쓰고, 명령어는 부록 D 에 정리했습니다

수업 일정과 프로젝트의 관계

주차	수업일	수업(강의자료)에서 배우는 것	프로젝트에서 하는 것
1주	9/4(금)	강의 소개	—
2주	9/11(금)	오픈소스소프트웨어의 기초(철학·라이선스·활용)	팀 구성, 역할, 팀 저장소 생성, 주제 후보
3주	9/18(금)	FOSS 혜택, 기여 준비(README·LICENSE·CONTRIBUTING·행동 수칙·이슈 트래커), Git 기초(GitHub 가입·토큰, Git Bash 커밋·무시)	계정·토큰·Git 설치, 실습 저장소(OSS), 주제 확정
4주	9/25 휴강	추석 연휴 (12/11 보강)	제안서 작성
5주	10/2(금)	프로젝트 제안서 발표	제안서 제출(10/1 18:00)·발표, 기본 파일 업로드

주차	수업일	수업(강의자료)에서 배우는 것	프로젝트에서 하는 것
6주	10/9 (온라인 대체)	프로젝트 찾기, 소스트리·VS Code, 소스트리로 커밋·푸시	담당 기능 이슈 등록, 팀 저장소 클론, 첫 커밋·푸시
7주	10/16(금)	사람들이 중요하다, 브랜치 만들기·체크아웃·병합	각자 feature/기능이름 브랜치에서 개발
8주	10/23(금)	중간고사	회의록·최소 기준 면제
9주	10/30(금)	기여하기, 병합 충돌 해결, 풀 리퀘스트·리뷰, 버전과 태그	첫 풀 리퀘스트·리뷰·병합, 충돌 실습, 브랜치 보호
10주	11/6(금)	프로젝트 중간발표	중간보고서 제출(11/5 18:00)·발표(MVP(최소 기능 제품) 시연)
11주	11/13(금)	둘 이상의 원격 저장소로 협업(포크·풀 리퀘스트)	타 팀 이슈 배정·Fork
12주	11/20(금)	실무 사례와 함께 Git 다루기(어멘드·체리 픽·리셋·리버트·스태시)	타 팀 풀 리퀘스트 제출, 기능 구현
13주	11/27(금)	자신만의 프로젝트 시작하기(필수 파일, 더 좋은 풀 리퀘스트, 병합·되돌리기, 브랜치 보호)	타 팀 풀 리퀘스트 병합 확인, 기능 동결, 문서 정비
14주	12/4(금)	CLI 환경에서 Git 활용하기	v1.0.0 태그·릴리스
4주차 보강	12/11(금)	최종발표 준비 기간(팀별)	새 폴더 실행 확인, 최종보고서·발표자료
15주	12/18(금)	프로젝트 최종발표	최종보고서 제출(12/17 18:00)·발표

- 읽는 순서: 1 장 → 3 장(로드맵) → 4 장(주제) → 6 장(준비하기) → 나머지는 필요한 시점에

1. 프로젝트 개요

1.1 한 줄 요약

[참고] 4명이 한 팀이 되어 GitHub에서 Python 프로젝트 1개를 한 학기 동안 만들고, **공개 저장소(v1.0.0 태그) + 중간·최종보고서 + 발표**로 제출합니다. 주제는 자유입니다.

팀	4인 1팀, 역할 4개(팀장 / 개발 리드 / 문서 담당 / 리뷰·품질 담당) — 코딩·풀 리퀘스트·리뷰는 4명 모두
언어·도구	Python, GitHub, Git·Git Bash, 소스트리, VS Code — 수업에서 쓰는 도구 그대로
일정	2주차(9/11) 팀 구성 → 5주차(10/2) 제안서 발표 → 9주차(10/30) 첫 풀 리퀘스트 → 10주차(11/6) 중간발표 → 13주차(11/27) 기능 동결 → 14주차(12/4) v1.0.0 → 15주차(12/18) 최종발표
결과물	동작하는 프로그램(핵심 기능 4개), README·LICENSE·CONTRIBUTING·행동 수칙, 회의록 10회, 중간보고서, 최종보고서, 발표 3회
평가	- 프로젝트 100 점 = 보고서·발표 40 + 활동 규칙 30 + 결과물 30. - 과목 성적의 50% (출결 10 / 중간고사 40 / 팀 프로젝트 50)

1.2 프로젝트 목적

- GitHub 기반 실무형 협업 능력 습득
- 오픈소스 문화 이해 및 기여 체험 — 문서·라이선스·행동 수칙을 갖춘 저장소 운영, 타 팀 저장소에 기여
- 코딩 실력 향상 및 기능 설계 역량 강화
- 팀워크와 커뮤니케이션 능력 향상
- 실질적인 GitHub 활동 기록(Contribution, 리뷰, PR 등) 축적 — 학기가 끝나면 내 프로필에 커밋·풀 리퀘스트·리뷰 기록이 남음

1.3 프로젝트 완성 기준 5 개

최종발표 때 아래 5개를 만족해야 하며, 이 5개가 결과물 점수(30점)의 기준입니다.

#	완성 기준	확인 방법
1	GitHub 공개 저장소에 v1.0.0 태그가 있고, README의 설치·실행 방법대로 새 PC에서 실행됨	저장소 태그, 다른 팀원의 재현
2	핵심 기능 4개(팀원 1인 1기능)가 동작하고 발표에서 시연	시연, README 기능 표
3	README, LICENSE, CONTRIBUTING, 행동 수칙(CODE_OF_CONDUCT) 파일이 있음	저장소 첫 화면
4	모든 기능이 브랜치 → 풀 리퀘스트 → 리뷰 → 병합으로 main에 들어감	Pull requests 목록

#	완성 기준	확인 방법
5	팀이 맡은 타 팀 이슈 1개를 풀 리퀘스트로 해결해 병합, 우리 저장소에 온 타 팀 풀 리퀘스트 병합, 회의록 10회 제출	타 팀 PR, LMS

1.4 제출 일정 한눈에 보기 (마감은 모두 수업일 전날 목요일 18:00)

제출물	마감	수업(발표)	형식·제출처
팀 명단	9/11(금) 수업 시간	2주차	수업 시간에 제출
제안서	10/1(목) 18:00	5주차 10/2 발표 10분 + 질의 5분	PDF, LMS 과제 탭. 발표자료는 발표 후 이메일
회의록 10회	10/1, 10/8, 10/15, 10/29, 11/5, 11/12, 11/19, 11/26, 12/3, 12/10 (목) 18:00	5주차~보강 주(12/11), 중간고사·최종발표 주 제외	PDF, LMS 과제 탭 (12장)
중간보고서	11/5(목) 18:00	10주차 11/6 중간발표 10분 + 질의 5분	PDF, LMS. 발표자료는 발표 후 이메일 (13장)
최종보고서	12/17(목) 18:00	15주차 12/18 최종발표 15분 + 질의 5분	PDF, LMS. 발표자료는 발표 후 이메일 (14장)

[주의] 휴강일: 9/25(추석 연휴, 12/11에 보강). 10/9(한글날)는 온라인 대체 수업이고, 12/11(금)은 4주차 보강으로 최종발표 준비 시간입니다. 수업이 없는 주에도 팀 활동(커밋·회의)은 계속되며, 회의는 온라인으로 해도 됩니다.

2. 팀 구성과 역할

2.1 팀 구성 규칙

- 2주차(9/11) 수업 시간에 4인 1팀으로 구성. 미리 팀을 꾸린 경우 수업 시간에 알리고, 팀이 없는 학생은 무작위 배정.
- 팀 이름은 영문 소문자·숫자·하이픈만 (예: team-busan) — 회의록 파일명과 저장소 이름에 사용
- 팀 구성 직후 할 일 3가지: 연락 채널 개설(카카오톡 또는 디스코드), 정기 회의 시간 확정, 각자 GitHub 아이디 공유

2.2 역할 4개

역할은 '조금 더 챙기는 사람'을 정하는 것입니다. 코딩·풀 리퀘스트·리뷰·이슈 작성은 4명 모두 같은 기준으로 평가되며, 역할만 맡고 코드를 쓰지 않는 경우는 인정되지 않습니다. 수업 3주차의 오픈소스 역할 구조로 보면 팀장이 리더·메인테이너, 나머지 팀원이 커미터에 해당합니다.

역할	챙기는 일	담당 기능
팀장	저장소 소유·설정, 회의 진행, 일정·이슈 정리, 교수와의 소통, 회의록·보고서 LMS 제출 확인	기능 1개
개발 리드	프로그램 구조(파일·모듈·데이터 형식) 결정, 공통 코드, 병합 시 기술적 판단	기능 1개 + 공통 코드
문서 담당	README·CONTRIBUTING·행동 수칙 관리, 회의록 작성, 타 팀 기여 창구	기능 1개
리뷰·품질 담당	풀 리퀘스트 리뷰가 48시간 안에 되도록 챙김, 버그 이슈 정리, 릴리스 전 실행 확인	기능 1개

2.3 팀 약속 (2주차, 부록 F 양식)

갈등이 생겼을 때 기준이 되는 5줄짜리 약속입니다. 저장소의 docs/team-agreement.md로 올립니다.

- 정기 회의: 주 1 회 60 분, 요일·시간 고정 (예: 매주 화 20:00 디스코드)
- 연락 응답: 채팅 24 시간 이내 / 풀 리퀘스트 리뷰 48 시간 이내
- 작업 전 이슈 등록, 하루 작업 끝에 푸시 (내 PC에만 두지 않기)
- 결정은 회의에서 다수결, 동률이면 개발 리드가 결정, 결과는 회의록에 기록
- 1 주 무응답 → 팀장이 연락 / 2 주 → 회의록에 기록하고 교수 상담

2.4 팀에 문제가 생겼을 때

- 참여하지 않는 팀원: 회의록 '활동 로그'에 사실만 기록(날짜·요청·응답 여부). 활동 규칙의 '각자' 항목은 개인별로 채점되므로 GitHub 기록이 곧 근거가 됨
- 팀원 이탈(수강 취소 등): 즉시 교수에게 이메일. 기능 4개 → 3 개로 조정
- 기술적으로 막힘: 이슈에 question 라벨로 등록 → 팀 채널 → 24 시간 안에 안 풀리면 교수 이메일 (16 장 질문 양식)

3. 15주 로드맵

아래 표가 이 프로젝트의 '지도'입니다. 매주 회의 첫 5분에 해당 주차를 함께 읽고 시작하세요. 개인별 체크리스트는 부록 A에 있습니다.

주차	수업일	수업 내용	이번 주 할 일	제출(마감)
2주	9/11(금)	OSS 기초	- 팀: 팀 구성·역할·팀장, 팀 약속, 채널, 주제 후보 3개(4장). - 팀장: GitHub 웹에서 공개 저장소 생성 + 협력자 초대(7.1~7.2) - 각자: GitHub 계정(6.2), 초대 수락	팀 명단(수업 시간)
3주	9/18(금)	기여 준비, GitHub 가입·토큰, Git 기초 (Git Bash)	- 각자: 토큰 발급, Git 설치·설정(6.2~6.3), 수업 실습 저장소(OSS) 완성 - 팀: 주제 확정, 기능 4개 배정(4.4 표 형식), 제안서 작성 시작(5장)	—
4주	9/25 휴강	추석 연휴 (12/11 보강)	팀: 제안서 완성, 첫 정기 회의 → 회의록 #1	제안서 PDF·회의록 #1: 10/1(목) 18:00
5주	10/2(금)	제안서 발표	팀: 발표 10분 + 질의 5분. 팀장: 기본 파일 업로드(7.3, GitHub 웹)	발표자료 (발표 후 이메일)
6주	10/9 (온라인 대체)	프로젝트 찾기, 소스 트리·VS Code, 소스트리로 커밋·푸시	- 각자: 담당 기능 이슈 3개 이상 등록(8.4, GitHub 웹) - 각자: 온라인 수업 후 소스트리 설치·GitHub 로그인(6.4), 팀 저장소 클론(7.4), docs/members/<이름>.md 자기소개 파일 → 커밋 → Push (첫 커밋) - 개발 리드: main.py 뼈대·공통 코드	회의록 #2: 10/8(목)
7주	10/16(금)	커뮤니티 예절, 브랜치 만들기·체크아웃·병합	각자: feature/기능이름 브랜치 생성 → 담당 기능의 첫 동작 코드 커밋 → 브랜치 Push (병합은 9주차 풀 리퀘스트로)	회의록 #3: 10/15(목)
8주	10/23(금)	중간고사	활동 최소 기준 면제. 시험 뒤 주말부터 브랜치 작업 재개	—(중간고사 주 회의록 없음)
9주	10/30(금)	기여하기, 병합 충돌 해결, 풀 리퀘스트, 태그	- 각자: 첫 풀 리퀘스트 → 팀원 1명 리뷰(Approve) → Merge pull request. 충돌 해결 실습(부록 D-2) - 팀장: main 브랜치 보호(7.5)	회의록 #4: 10/29(목)
10주	11/6(금)	중간발표	- 팀: 중간보고서(13장) 제출·발표 10분 + 질의 5분(MVP 시연 — 기능 2개는 첫 풀 리퀘스트로 병합) - 문서 담당: 타 팀 기여용 good first issue 2개 이상 등록(11.2)	중간보고서·회의록 #5: 11/5(목) 18:00, 발표자료(이메일)
11주	11/13(금)	포크, 포크에서 풀 리퀘스트, 승인·병합	- 팀: 매칭된 상대 팀 이슈 1개 선택·배정 요청 → 팀장 Fork → 팀원 협력자 등록(11.3) - 팀: 우리 저장소에 온 배정 요청 응답	회의록 #6: 11/12(목)
12주	11/20(금)	실무 사례 Git(어멘드·체리 픽·리셋·리버트·스테시)	팀: 상대 팀에 풀 리퀘스트 제출(~11/26). 우리 저장소에 온 풀 리퀘스트 리뷰. 기능 3~4 구현, 수업의 어멘드·리버트·스테시로 실수 정리(8.8)	회의록 #7: 11/19(목)

주차	수업일	수업 내용	이번 주 할 일	제출(마감)
13주	11/27(금)	프로젝트 공개 파일, 더 좋은 풀 리퀘스트, 되돌리기, 브랜치 보호	- 팀: 상대 팀 리뷰 반영 → 병합 확인(~12/3). 우리 저장소에 온 풀 리퀘스트 병합 - 팀: 기능 동결 — 새 기능 금지, 버그 수정·문서만. README·CONTRIBUTING·행동 수칙·LICENSE 최종 정비(14.2)	회의록 #8: 11/26(목)
14주	12/4(금)	Git Bash 명령어	팀: 완성 기준 5개(1.3) 점검 → v1.0.0 태그·릴리스(14.1)	회의록 #9: 12/3(목)
4주차 보강	12/11(금)	최종발표 준비(팀별)	팀: 새 폴더에서 클론·실행 확인, 최종보고서·발표자료 작성, 시연 녹화. 수업 시간은 팀별 준비 시간	회의록 #10: 12/10(목)
15주	12/18(금)	최종발표	팀: 발표 15분 + 질의 5분, 전원 발언	최종보고서: 12/17(목) 18:00 , 발표자료(이메일)

3.1 단계별 목표

단계	주차	끝났을 때의 상태
계획·준비	2~6주	제안서 발표 완료. 4명 모두 소스트리로 팀 저장소를 클론해 커밋·푸시 1회 이상. 담당 기능 이슈 등록
개발	7~10주	각자 브랜치에서 기능을 만들고 풀 리퀘스트로 합침. 각자 충돌 해결 1회. 중간발표에서 MVP(기능 2개) 시연
기여·완성	11~13주	팀이 상대 팀 이슈 1개를 풀 리퀘스트로 해결·병합. 우리 저장소에 온 풀 리퀘스트 병합. 기능 4개 완성
마무리	13~15주	기능 동결 → 문서 정비 → v1.0.0 태그·릴리스 → 최종보고서·발표

3.2 주차별 상세

2~5 주차: 팀 구성과 제안서 (수업: OSS 기초 → Git 기초 → 발표)

- 2 주차(9/11): 팀장·역할 결정(2.2), 팀 약속 작성(부록 F), 채널·회의 시간 확정. 주제 후보 3 개를 4 장 기준으로 비교
- 2 주차: 팀장이 GitHub 웹에서 **공개** 저장소 생성(README·.gitignore(Python)·MIT 포함) → 팀원 3 명 협력자 초대 → 팀원은 이메일에서 수락 (7.1~7.2). GitHub 계정이 없는 팀원은 6.2 절대로 먼저 만들
- 3 주차(9/18) 수업에서 GitHub 가입·토큰·Git 설치·Git Bash 실습(OSS 저장소)을 마칩. 이 실습 저장소가 활동 규칙 '저장소 기여 3 개' 중 첫 번째
- 3 주차 회의: 주제 확정, 기능 4 개로 나누어 1 인 1 기능 배정(4.4 표), 제안서(5 장 7 항목) 분담 작성
- 추석 연휴 주(9/21~27): 제안서 완성 → **10/1(목) 18:00** PDF 제출, 회의록 #1 제출. 5 주차(10/2) 발표 10 분 + 질의 5 분
- 5 주차 이후~10/8: 팀장은 GitHub 웹의 **Add file** 로 CONTRIBUTING·행동 수칙·requirements.txt·main.py 기본 파일 업로드(7.3). 각자 담당 기능을 이슈 3 개 이상으로 등록(8.4) — GitHub 웹에서 하므로 Git 없이 가능

6~9 주차: 첫 커밋 → 브랜치 → 첫 풀 리퀘스트 (수업: 소스트리 → 브랜치 → 충돌·풀 리퀘스트)

1. 6 주차(10/9 온라인 수업)에서 소스트리·VS Code 를 설치하고 GitHub 계정을 연결함. 팀 저장소를 소스트리 Clone 으로 내려받아 docs/members/<이름>.md 에 자기소개 3 줄을 쓰고 커밋 → Push. 누군가 먼저 올렸다면 Pull 후 Push. 개발 리드는 main.py 뼈대(메뉴 4 개가 '준비 중' 출력)를 올림. 6 주차까지는 main 에 직접 푸시해도 됨 — 한 명씩 순서대로 Pull → 커밋 → Push
2. 7 주차(10/16) 수업의 실습 브랜치 규칙 4 개가 그대로 팀 규칙(8.1). 각자 소스트리 브랜치로 feature/기능이름 생성 → 담당 기능의 첫 동작 코드 커밋 → 브랜치 Push. 아직 병합하지 않음
3. 8 주차(10/23) 중간고사: 회의록 없음, 커밋·풀 리퀘스트·이슈 최소 기준 면제. 시험이 끝난 주말부터 브랜치 작업 재개
4. 9 주차(10/30) 수업 직후: GitHub 의 Compare & pull request 로 첫 풀 리퀘스트 → Reviewers 에 팀원 지정 → 리뷰어가 Approve → Merge pull request. 이때부터 main 은 풀 리퀘스트로만 바뀜. 팀장은 7.5 절대로 main 브랜치 보호를 켜. 2 인 1 조로 부록 D-2 충돌 해결 실습. 중간발표(11/6) 전까지 기능 2 개를 병합해 MVP 완성

10~13 주차: 중간발표와 타 팀 기여 (수업: 중간발표 → 포크 → 실무 Git → 프로젝트 공개)

1. 10 주차(11/6) 중간발표: 중간보고서 11/5(목) 18:00 제출 → 발표 10 분 + 질의 5 분, MVP(기능 2 개) 실제 시연(13 장). 문서 담당은 타 팀이 기여할 good first issue 2 개 이상 등록(11.2)
2. 11 주차(11/13) 수업의 '개발자 C' 역할을 팀 단위로 함: 매칭표의 상대 팀 good first issue 중 1 개를 골라 댓글로 배정 요청 → 팀장이 Fork → fork 에 팀원 협력자 등록 → 담당을 나눠 작업(11.3). 우리 저장소에 온 배정 요청에는 24 시간 안에 응답. 상대 팀 풀 리퀘스트는 11/26 까지 제출
3. 12~13 주차(11/20, 11/27): 상대 팀 리뷰 반영 → 12/3 까지 병합 확인. 우리 저장소에 온 풀 리퀘스트는 48 시간 안에 리뷰·병합. 수업에서 배운 어멘드·리버트·스테시로 실수 정리(8.8). 기능 4 개 완성 목표

13~15 주차: 동결·릴리스·최종발표 (수업: 프로젝트 공개 → Git Bash → 준비 → 발표)

1. 13 주차(11/27): 기능 동결. 수업 13 주차 '프로젝트에 꼭 포함할 파일'(README·사용자 가이드·CONTRIBUTING·행동 수칙·라이선스)을 기준으로 문서 최종 정비(14.2)
2. 14 주차(12/4): 완성 기준 5 개(1.3) 점검 → 소스트리 태그로 v1.0.0 생성·Push(수업 9 주차 방법) → GitHub Releases 에 릴리스 노트(14.1)
3. 4 주차 보강(12/11): 수업 시간이 팀별 준비 시간. 팀원 1 명이 새 폴더에 클론해 README 대로 실행 확인 → 최종보고서·발표자료 작성 → 시연 녹화
4. 15 주차(12/18): 최종보고서 12/17(목) 18:00 제출 → 발표 15 분 + 시연 + 질의 5 분 → 발표자료 이메일 제출

4. 주제 정하기 (2~3 주차)

주제는 자유입니다. 이 프로젝트의 목적은 '대단한 프로그램'이 아니라 GitHub에서 4명이 함께 만들고 기록을 남기는 경험이므로, 확실하게 동작하는 주제가 좋습니다.

4.1 좋은 주제 고르는 질문 3개

질문	이유
4명이 기능 4개로 나눌 수 있는가?	1인 1기능이어야 각자 브랜치·풀 리퀘스트·커밋 기록이 고르게 남음
첫 동작을 1~2주 안에 볼 수 있는가?	7주차에 브랜치를 만들자마자 '뭔가 되는' 코드를 올릴 수 있어야 함
API 키·외부 서버·큰 데이터 없이도 시작할 수 있는가?	샘플 데이터 파일 하나로 시작하고, 외부 연동은 나중에 붙이는 것이 안전

4.2 규모

핵심 기능	4개(1인 1기능) + 공통 코드(저장·읽기, 메뉴). 5개 이상이면 제안서에서 줄이도록 요청
기능이란	사용자가 메뉴 하나·명령 하나로 쓰는 단위. 예: '거래 추가', '월별 통계', '예산 경고', '그래프 저장'
MVP(중간발표, 10주차)	기능 2개가 실제 데이터로 동작 + README대로 실행됨
기능 동결(13주차)	이후 새 기능 금지. 남은 기간은 버그·문서·릴리스
코드량	전체 300~1,000줄 정도이나 넘겨도 무방함

4.3 유형별 예시

분류	예시 주제	도구(pip)	난이도·비고
CLI 도구	가계부, 할 일 관리, 폴더 정리 자동화, 날짜·단위 계산기, 학점 계산기	표준 라이브러리(csv, json, argparse)	★ 쉬운 난이도
데이터 분석	부산 공공데이터(교통·대기질·인구) 분석 리포트, 시각화 대시보드	pandas, matplotlib	★★ 데이터를 먼저 CSV로 확보(공공 데이터포털 data.go.kr)
간단한 앱	메모장, 미니 게시판, 설문·투표, 시간표 관리	streamlit(권장, Python만으로 화면) 또는 flask	★★ 로그인·회원가입은 만들지 않기
게임	텍스트 기반 퀴즈, 가위바위보, 끝말잇기, 타자 연습, 2048	표준 라이브러리, pygame	★★ 텍스트 게임 권장
도구·자동화	GitHub 활동 통계 스크립트, 문서 생성기, 디스코드 알림 봇	requests, discord.py	★★★★ 토큰은 저장소에 올리지 않기 (9.5)

4.4 예시: 가계부 CLI 'moneylog' — 기능 나누기

제안서 3번 항목(주요 기능 설계)은 이 표 형식으로 씁니다.

구분	담당	내용	완료 조건
공통	개발 리드	거래 데이터 형식(날짜·금액·분류·메모), storage.py(CSV 저장·읽기), main.py 메뉴	메뉴 4개가 뜨고 각 기능을 호출
기능 1	팀장	add — 거래 입력·입력 검증	잘못된 입력에 안내 메시지, 정상 입력은 CSV에 저장
기능 2	팀원 B	summary — 월별 합계, 분류별 합계	샘플 데이터로 계산 결과가 맞음
기능 3	팀원 C	budget — 분류별 예산 설정·초과 경고	예산 초과 시 경고 출력
기능 4	팀원 D	chart — 분류별·월별 그래프 PNG 저장(matplotlib)	docs/에 PNG 생성, README에 이미지

이슈 예: [기능] add 입력 검증, [기능] CSV 저장 시 utf-8 고정, [기능] summary 분류별 합계, [버그] 금액에 쉼표가
있으면 오류, [문서] README 사용 예시

4.5 피해야 할 주제

- 로그인·회원가입·결제 중심인 웹 서비스 — 인증에 시간을 다 쓰고 기능이 안 남음
- 실시간 크롤링에 전적으로 의존 — 대상 사이트가 바뀌면 발표 날 멈춤. CSV 데이터 + 갱신은 선택 기능으로
- 유료 API가 꼭 필요한 것 — 비용·키 공유 문제. 무료 범위·샘플 응답으로 대체

5. 프로젝트 제안서 (5 주차 10/2 발표)

5.1 항목 7 개

#	항목	들어갈 내용	분량
1	팀 소개	팀명, 팀원 이름·역할, 팀원별 GitHub 아이디, 저장소 주소	1쪽
2	프로젝트 주제 및 개요	프로젝트 이름, 해결하고자 하는 문제, 간단한 기능 개요 또는 핵심 아이디어	1쪽
3	주요 기능 설계	기능 4개 + 공통(4.4 표 형식: 담당·내용·완료 조건), 기능별 브랜치 이름 예시 (feature/...)	1~2쪽
4	개발 및 협업 일정 계획	주차별 표(6~15주차). MVP(10주차 중간발표), 타 팀 기여(11~13주차), 기능 동결(13주차), v1.0.0(14주차) 표시	1쪽
5	GitHub 협업 계획	브랜치 전략(8.1), 커밋 메시지 규칙, 풀 리퀘스트·리뷰 규칙, 이슈 등록 방식, 회의록·문서 위치, 라이선스와 행동 수칙	1쪽
6	예상 결과물	최종 형태(CLI/웹/봇 등), 이후 개선 또는 외부 기여 계획	1쪽
7	기대 효과 및 목표	협업 경험·GitHub 활용 능력 향상, OSS/코딩 역량 인증제 대비, 팀워크·문서화 경험 — 팀의 목표를 자기 말로	0.5쪽
+	기타	참고문헌(해당 시), 기타 필요한 사항	—

5.2 제출과 발표

제출 마감	10/1(목) 18:00 LMS 과제 탭
형식	PDF. 표지·목차 + 1~7 항목 + 기타. 파일명 [제안서] 팀명.pdf
발표	5주차 10/2(금) 수업 시간, 팀당 10분 + 질의 5분. PPT 발표자료 준비, 팀원 전원 1회 이상 발언
발표자료	발표 후 당일 이메일 제출: shoh0320@pknu.ac.kr, 제목 [OSS 제안서] 팀명
평가	제안서 10점 + 발표 5점 — 기준은 15.1절

6. 준비하기: 계정과 도구 (3~6 주차, 각자)

필요한 것은 수업 3주차(GitHub·Git)와 6주차(소스트리·VS Code, 10/9 온라인)에서 설치하는 것과 같습니다. 수업 시간에 따라 했다면 6.6절 체크리스트만 확인하세요. 아래는 수업 자료를 요약한 것이며, 자세한 화면은 해당 주차 강의 자료를 보면 됩니다.

6.1 필요한 것

항목	용도	수업	비고
GitHub 계정	저장소·이슈·풀 리퀘스트·리뷰	3주차	학교 이메일(@pukyong.ac.kr 또는 @pknu.ac.kr)을 계정에 추가해 두면 GitHub Student Developer Pack 신청 가능(선택)
개인 액세스 토큰	Git Bash에서 push할 때 비밀번호 대신 사용	3주차	생성 직후 복사해 메모장에 보관 (다시 볼 수 없음)
Git (Git Bash)	버전 관리 도구	3주차	git-scm.com 설치 파일
소스트리	Git을 버튼으로 다루는 GUI	6주차	sourcetreeapp.com. 이 안내서의 기본 도구
VS Code	코드·문서 편집	6주차	code.visualstudio.com + Korean Language Pack, Python 확장
Python	프로그램 실행	2주차 예시	python.org

6.2 GitHub 계정과 토큰 (수업 3 주차)

1. github.com → Sign up → 이메일·비밀번호·아이디 입력 → 이메일로 온 코드 입력 → 가입 완료. 아이디는 포트폴리오에 남으므로 알아보기 쉬운 영문으로
2. 프로필 → Settings → Emails 에서 학교 이메일 추가(선택). 같은 화면의 'Keep my email addresses private'가 켜져 있으면 숫자+아이디@users.noreply.github.com 주소가 보이는데, Git·소스트리에 이메일을 넣을 때 이 주소를 써도 됨
3. 토큰: Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic) → Note 에 OSS 프로젝트, Expiration 은 12 월 이후, 권한(scope)에서 repo 체크 → Generate token → 복사해 보관

[왜?] 커밋이 내 GitHub Contribution 그래프에 집계되려면 Git과 소스트리에 입력한 이메일이 GitHub 계정의 이메일(또는 noreply 주소)과 같아야 합니다. 6.3절과 6.4절에서 같은 이메일을 넣으세요.

6.3 Git 설치와 첫 설정 (수업 3 주차)

- Windows: git-scm.com/downloads → Download for Windows → 설치 파일 실행 → 계속 Next → Finish. 시작 메뉴에서 Git Bash 실행

- macOS: 터미널(Spotlight 에서 'terminal')에서 `git --version` 입력 → 설치 안내 창이 뜨면 설치 (Xcode Command Line Tools)

Git Bash(또는 macOS 터미널)에서 내 정보를 등록합니다. 수업 3주차 '첫 번째 커밋 만들기'와 같은 명령입니다.

```
git config --global user.name "홍길동"
git config --global user.email "GitHub에 등록한 이메일"
git config --global core.editor "code --wait"      # 커밋·병합 메시지 편집기를 VS Code로 (수업 14주차 권장)
git --version                                     # git version 2.4x 가 나오면 성공
```

[TIP] Git Bash에서 처음 `git push`를 하면 GitHub 로그인 창이 뜹니다. 브라우저로 로그인하거나, 비밀번호 칸에 6.2절의 토큰을 붙여 넣으면 됩니다. 한 번 로그인하면 Windows가 기억합니다.

6.4 소스트리와 VS Code (수업 6 주차)

1. `sourcecodeapp.com` → 운영체제에 맞는 파일 다운로드 → 설치. Bitbucket 로그인 화면은 건너뛰기, Mercurial 체크 해제, 이름·이메일은 **GitHub과 같게** 입력, SSH 키는 **아니오**
2. 소스트리 → Remote → 계정 추가 → 호스팅 서비스 GitHub → OAuth 토큰 새로고침 → 브라우저에서 GitHub 로그인 → 확인. 원격 저장소 목록에 내 저장소가 보이면 성공
3. `code.visualstudio.com` → 다운로드·설치. 왼쪽 확장에서 **Korean Language Pack** 과 **Python(Microsoft)** 설치
4. 소스트리 상단의 **터미널** 버튼을 누르면 저장소 폴더에서 Git Bash가 열립니다. `pip·python` 명령은 여기서 실행하면 됩니다

6.5 Python 설치

1. `python.org/downloads` → 최신 안정 버전(3.12~3.14) 설치. Windows 는 설치 첫 화면의 '**Add python.exe to PATH**' 체크가 중요
2. 확인: Git Bash 또는 CMD 에서 `python --version`, `pip --version`
3. 라이브러리 설치: 수업 2 주차 예시처럼 `pip install 이름`. 팀 프로젝트에서는 `requirements.txt`에 적어 두고 `pip install -r requirements.txt`로 한 번에 설치(9.3)

[참고] Python 3.15가 출시될 예정이지만, 라이브러리 호환 문제를 피하기 위해 학기 중에는 설치한 버전을 바꾸지 마세요.

6.6 준비 완료 체크리스트 (6 주차까지)

확인 항목	완료
GitHub 계정, 토큰 보관, (선택) 학교 이메일 등록	☐
git --version 정상, user.name·user.email 등록(이메일은 GitHub과 동일)	☐
수업 3주차 실습 저장소(OSS)에 커밋 2개 이상 push 완료	☐
소스트리 설치 + GitHub 계정 연결(Remote에 내 저장소가 보임)	☐
VS Code + Korean Language Pack + Python 확장	☐
python --version 정상(3.12 이상)	☐
팀 저장소 초대 수락 → 소스트리로 클론 완료(7.4)	☐

7. 팀 저장소 만들기 (2~6 주차)

저장소 생성과 협력자 등록은 GitHub 웹에서 하므로 2주차에 바로 할 수 있습니다. 클론은 6주차(10/9 온라인 수업)에 소스트리를 배운 뒤, 브랜치 보호는 9주차에 풀 리퀘스트를 시작하면서 겁니다.

7.1 저장소 생성 (팀장, 2 주차)

1. GitHub → 오른쪽 위 + → New repository
2. Owner: 팀장 계정 / Repository name: **프로젝트 이름** (영문 소문자·숫자, 예: moneylog, busanair)
3. Description: 한 줄 소개 (한글 가능)
4. Public 선택
5. Add a README file 체크 / .gitignore template → Python / Choose a license → MIT License
6. Create repository

[왜?] Public이어야 하는 이유: 타 팀이 Fork해서 기여하려면 볼 수 있어야 하고, 공개 저장소의 활동만 프로필의 Contribution 그래프에 표시되며, 브랜치 보호 규칙도 무료 계정에서는 공개 저장소에만 제공됩니다. 비공개 저장소는 인정되지 않습니다.

[참고] 라이선스는 수업 2주차에서 배운 방임형 라이선스 MIT를 기본으로 합니다. 다른 OSI 승인 라이선스를 고려하면 제안서에 이유를 한 줄 적으세요. 라이선스 파일이 없으면 오픈소스가 아닙니다(수업 3주차).

7.2 팀원 협력자(Collaborator) 등록 (수업 11 주차 '협력자 등록'과 같은 화면)

1. 저장소 → Settings → Collaborators → Add people → 팀원 GitHub 아이디 입력 → Add
2. 팀원은 GitHub 알림 또는 이메일에서 Accept invitation. 초대는 7 일 뒤 만료되므로 당일 수락
3. 확인: Collaborators 목록에 팀원 3 명이 표시되면 완료

7.3 기본 파일 넣기 (팀장, 5 주차 이후, GitHub 웹에서)

소스트리를 배우기 전이므로 GitHub 웹에서 만듭니다. 저장소 → Add file → Create new file → 파일 이름 입력(폴더는 docs/meetings/README.md처럼 /로) → 부록 C 내용 붙여넣기 → Commit changes. 파일 이름의 moneylog는 여러분의 프로젝트 이름으로 바꿉니다.

```

moneylog/
├─ README.md          # 소개·설치·실행·팀원 (부록 C-1) - 생성 시 자동 생성된 파일을 고쳐 씀
├─ LICENSE            # MIT (생성 시 자동)
├─ .gitignore         # Python 템플릿 (생성 시 자동)
├─ CONTRIBUTING.md    # 기여 방법 (부록 C-2)
├─ CODE_OF_CONDUCT.md # 행동 수칙 (부록 C-3, 수업 3·13주차 '필수 파일')
├─ requirements.txt    # 사용하는 라이브러리 목록 (부록 C-4)
├─ main.py            # 프로그램 시작 파일 (부록 C-5)
└─ docs/
   └─ meetings/README.md # 회의록 폴더 (내용: '회의록 저장 폴더')
   └─ members/          # 6주차에 각자 자기소개 파일 추가
   └─ team-agreement.md # 팀 약속 (부록 F)

```

7.4 소스트리로 클론 (전원, 6 주차)

1. GitHub 팀 저장소 → 초록색 Code 버튼 → HTTPS 주소 오른쪽의 복사 버튼
2. 소스트리 → Clone → 첫 칸에 주소 붙여넣기 → 둘째 칸 탐색으로 내 PC 의 폴더 선택(예: 바탕 화면/Programming/moneylog) → 클론
3. 새 탭에 저장소가 열리고 History 에 커밋이 보이면 성공. VS Code → 파일 → 폴더 열기로 같은 폴더를 엽니다
4. 첫 커밋: VS Code 에서 docs/members/<네이름>.md 파일을 만들어 자기소개 3 줄 저장 → 소스트리 커밋 → 파일 옆 +로 스테이지 → 메시지 홍길동 자기소개 추가 → -에 바뀐 내용 즉시 무시 체크 → 커밋 (수업 6 주차 실습과 동일)

[주의] Push가 거부되면(다른 팀원이 먼저 올림) 소스트리 상단 Pull → Pull 버튼 → 다시 Push. 여러 명이 main 에 직접 올리는 것은 6주차까지만이며, 7주차부터는 각자 브랜치에서 작업합니다.

7.5 main 브랜치 보호 (팀장, 9 주차) — 수업 13 주차 '브랜치 보호하기'와 동일

풀 리퀘스트를 시작하는 9주차에 컵니다. 커고 나면 누구도(팀장 포함) main에 직접 커밋할 수 없고, 리뷰 승인 1개 없이는 병합할 수 없습니다. 실수로 main을 망가뜨리는 사고를 막아 주는 안전장치입니다.

1. 저장소 → Settings → 왼쪽 Branches → Add classic branch protection rule
2. Branch name pattern 에 main 입력
3. Require a pull request before merging 체크 → 그 아래 Require approvals 체크(숫자 1)
4. 맨 아래 Do not allow bypassing the above settings 체크 → Create

확인: 소스트리에서 main에 직접 커밋해 Push하면 오류가 나고, 풀 리퀘스트는 승인 1개가 있어야 Merge pull request 버튼이 켜집니다. 본인 풀 리퀘스트는 본인이 승인할 수 없습니다.

7.6 선택: 라벨과 Project 보드

- 라벨은 GitHub 기본 라벨을 사용: enhancement(기능), bug, documentation, question, good first issue(타 팀 기여용)
- Project 보드(Projects 탭 → New project → Board)는 할 일을 Todo / In Progress / Done 으로 보고 싶을 때 씁니다. 필수는 아니며, 이슈 목록만으로도 충분합니다

저장소 준비 체크	담당	주차	완료
공개 저장소 + README + LICENSE(MIT) + .gitignore(Python)	팀장	2주	☐
팀원 3명 협력자 등록·수락	팀장·팀원	2주	☐
기본 파일 6종(CONTRIBUTING, 행동 수칙, requirements.txt, main.py, docs/meetings, 팀 약속)	팀장	5주	☐
담당 기능 이슈 각자 3개 이상	전원	5~6주	☐
전원 소스트리 클론 + 자기소개 파일 커밋·Push	전원	6주	☐
main 브랜치 보호 규칙	팀장	9주	☐

8. 협업 루틴 — 매주 반복하는 8 단계

7주차부터 최종발표까지 매주 같은 순서를 반복합니다: 이슈 → 브랜치 → 커밋 → 푸시 → 풀 리퀘스트 → 리뷰 → 병합 → 정리. 수업 7·9주차 실습을 팀 저장소에서 그대로 하는 것입니다. 이 장을 프린트해서 모니터 옆에 두세요.

8.1 브랜치 규칙 (수업 7 주차 '실습 브랜치 규칙'과 동일)

1. `main` 브랜치에는 직접 커밋을 올리지 않는다 (동시에 작업하면 꼬인다)
2. 기능 개발을 하기 전에 `main` 브랜치 기준으로 새로운 브랜치를 만든다
3. 브랜치 이름은 `feature/기능이름` 형식으로 하고, 한 브랜치에는 한 명만 커밋한다
4. 기능 개발이 끝나면 풀 리퀘스트로 `main` 에 합친다 (9 주차부터는 승인 1 개가 있어야 병합됨)

브랜치	쓰임	예
<code>main</code>	항상 실행되는 상태의 코드. 풀 리퀘스트로만 바뀜	—
<code>feature/기능이름</code>	기능 개발 (수업 7주차 규칙. 12주차 예시의 <code>feat/</code> 도 같은 뜻 — 팀에서 하나로 통일)	<code>feature/add-record</code> , <code>feature/monthly-summary</code>
<code>fix/문제이름</code>	버그 수정	<code>fix/csv-encoding</code>
<code>docs/내용</code>	문서만 수정	<code>docs/readme-install</code>

브랜치 이름은 영문 소문자와 하이픈만 씁니다. 브랜치 하나의 수명은 1주가 적당하며, 길어지면 풀 리퀘스트를 나누세요.

8.2 8 단계 루틴 (소스트리 기준)

단계	소스트리·GitHub에서 하는 일	수업
1. 이슈	GitHub Issues에서 내가 할 일의 이슈를 확인(없으면 만들, 8.4). 이슈 번호를 기억 (예: #12)	3주차
2. 최신화	소스트리 사이드바 <code>main</code> 더블클릭(체크아웃) → 상단 Pull	6주차
3. 브랜치	상단 브랜치 → 현재 브랜치가 <code>main</code> 인지 확인 → 새 브랜치 이름 <code>feature/기능이름</code> → 새 브랜치 체크아웃 체크 → 브랜치 생성	7주차
4. 코딩·커밋	VS Code로 수정·저장 → 소스트리 커밋 → 바뀐 파일을 +로 스테이지 → 메시지(8.3) → 커밋. 의미 있는 단위마다 반복 (하루에 여러 번)	6주차
5. 확인·푸시	프로그램을 실행해 동작 확인 → 소스트리 Push → 내 브랜치 체크 → Push (또는 커밋 창에 바뀐 내용 즉시 푸시 체크)	6~7주차
6. 풀 리퀘스트	GitHub 저장소에 뜨는 Compare & pull request → base <code>main</code> ← compare 내 브랜치 확인 → 제목·설명(8.5) → 오른쪽 Reviewers에서 팀원 1명 지정 → Create pull request	9주차
7. 리뷰·병합	리뷰어가 Files changed 확인 → Review changes → Approve (고칠 것이 있으면 Request changes). 수정은 같은 브랜치에 커밋·Push하면 풀 리퀘스트에 자동 반영. 승인되면 작성자가 Merge pull request → Confirm merge	9주차
8. 정리	소스트리 패치(그래프 갱신) → <code>main</code> 체크아웃 → Pull → 다 쓴 브랜치 우클릭 → 삭제. 이슈는 풀 리퀘스트 설명에 Closes #12를 적었으면 자동으로 닫힘	9주차

[TIP] 하루 작업이 끝나면 풀 리퀘스트가 준비되지 않았어도 브랜치를 Push해 두세요. PC가 고장 나도 코드가 남고, 팀원이 진행 상황을 볼 수 있습니다. Git Bash 명령으로 같은 일을 하고 싶으면 부록 D-1의 대응표를 보세요 (수업 14주차).

8.3 커밋 메시지

- 한 줄, 한글로, '무엇을 했는지'가 보이게. 수업 6주차 예: OSS 혜택 및 할 일 목록 추가
- 권장 형식: 기능 설명 (#이슈번호) — 예: 거래 입력 기능 추가 (#12), 빈 파일 읽을 때 오류 수정 (#31)
- 수업 13주차의 분류 접두어(feat: 새 기능, fix: 버그 수정, docs: 문서, refactor: 구조 정리, test:, chore:)를 앞에 붙이면 이력이 더 읽기 쉬움 (선택)
- 커밋 1개 = 변경 1가지. 30분~2시간 단위가 적당. 수정, 최종, asdf 같은 메시지는 금지
- 메시지를 잘못 썼거나 파일을 빠뜨렸으면 새 커밋 대신 수업 12주차의 마지막 커밋 정정(어멘드)으로 고침 — Push 전에만

8.4 이슈

- 작업 전에 이슈를 먼저 만듭니다 (수업 3주차 '이슈 트래킹', 9주차 '이슈 트래커 검토'). 이슈 없는 풀 리퀘스트는 리뷰어가 '이슈 먼저 만들어 주세요'로 되돌림
- 제목: [기능] 월별 합계 계산, [버그] 한글 CSV 깨짐, [문서] 설치 방법 보강, [질문] 저장 형식 JSON vs CSV
- 본문 3줄: 목적(왜 필요한가) / 할 일(2~5개 체크리스트) / 완료 조건(무엇이 되면 닫는가)
- 오른쪽에서 Assignees(담당자)와 Labels(enhancement·bug·documentation·question)를 지정
- 크기: 이슈 1개 = 1~3일 분량. 담당 기능은 이슈 3~5개로 나눔
- 닫기: 풀 리퀘스트 설명에 Closes #번호를 적으면 병합 시 자동으로 닫힘

8.5 풀 리퀘스트 (수업 9·13주차)

항목	기준
단위	풀 리퀘스트 1개 = 이슈 1개. 한 번에 여러 기능을 담지 않기 (수업 13주차 '잘 쪼갬 PR')
제목	커밋 메시지처럼 한 줄. 예: 거래 입력 기능 추가 (#12)
설명	3줄: 무엇을 바꿨나 / 왜(Closes #12) / 어떻게 확인했나(실행 방법). 화면이 바뀌면 스크린샷
리뷰어	작성자 외 팀원 1명 이상을 Reviewers에 지정. 돌아가며 맡아서 한 사람에게 몰리지 않게
병합	Approve 1개 + 충돌 없음 → 작성자가 Merge pull request(기본 방식 'Create a merge commit') → Confirm merge
기한	리뷰 요청 후 48시간 안에 리뷰. 늦어지면 팀장이 다른 리뷰어 지정

8.6 리뷰하는 법 (수업 7 주차 '사람들이 중요하다', 9 주차 '검토·개정·협업')

리뷰는 '틀린 곳 찾기'가 아니라 팀 코드를 함께 좋게 만드는 대화입니다. 15~20분이면 충분합니다.

1. 풀 리퀘스트 설명과 연결된 이슈의 완료 조건을 읽음
2. Files changed 탭에서 바뀐 부분을 읽고, 줄 번호 옆 +로 코멘트. 가능하면 브랜치를 소스트리로 체크아웃해 직접 실행해 봄
3. Review changes → Approve(이대로 병합 가능) / Request changes(고친 뒤 다시 봄) / Comment(의견만)

확인할 것	질문
동작	이슈의 완료 조건을 만족하나? 실행해 보니 되나? 잘못된 입력에는 어떻게 되나?
읽기	함수·변수 이름만 보고 역할을 알 수 있나? 너무 긴 함수는 없나?
문서	사용법이 바뀌었으면 README도 바뀌었나? 새 라이브러리가 requirements.txt에 있나?
안전	토큰·비밀번호·개인정보·큰 데이터 파일이 들어 있지 않나?

코멘트 예: [필수] 파일을 열고 닫지 않아요. with open(...) 으로 바꿔 주세요. / [제안] 이 반복문은 sum()으로 한 줄이 됩니다. / [질문] 날짜가 없는 행은 왜 건너뛰나요? / [칭찬] 오류 메시지가 친절하네요. — [필수]는 3개 이내로, 사람이 아니라 코드에 대해 말합니다.

[참고] 내용 없는 'LGTM'만 남긴 승인은 활동 규칙의 리뷰로 인정되지 않습니다. 실행해 봤다는 한 줄이나 구체적인 코멘트 1개면 충분합니다.

8.7 충돌(Conflict) 해결 — 수업 9 주차 '충돌 병합 해결하기'와 동일

두 사람이 같은 파일의 같은 부분을 고쳤을 때 GitHub 풀 리퀘스트에 'This branch has conflicts that must be resolved'가 뜹니다. 수업에서 배운 대로 내 브랜치에 main을 먼저 병합해 충돌을 해결한 뒤 Push하면 풀 리퀘스트의 충돌 표시가 사라집니다.

1. 소스트리 사이드바 main 체크아웃 → Pull (원격의 최신 main 을 받음)
2. 내 브랜치(feature/...) 체크아웃 → History 에서 main 의 최신 커밋 우클릭 → 병합 → 확인 → 충돌 경고가 뜨면 닫기
3. 커밋하지 않은 변경 사항을 선택하면 충돌 난 파일이 스테이지 아래에 보임
4. VS Code 에서 그 파일을 열면 <<<<<<, =====, >>>>>>로 두 버전이 표시됨. 위쪽은 내 브랜치, 아래쪽은 main 의 내용. 충돌 부분 위에 뜨는 메뉴(현재 변경 사항 수락 / 수신 변경 사항 수락 / 두 변경 사항 모두 수락)를 고르거나 손으로 정리해 표시 줄을 지우고 저장
5. 프로그램을 실행해 정상인지 확인 → 소스트리에서 파일을 스테이지에 올림 → 커밋 (메시지 자동 입력, 그대로 두어도 됨) → Push
6. GitHub 풀 리퀘스트에서 충돌 표시가 사라졌는지 확인 → 리뷰 → 병합

- 충돌을 줄이는 습관: 작업 시작 전과 풀 리퀘스트 올리기 전에 main 을 Pull 해서 내 브랜치에 병합 / 같은 파일을 동시에 고칠 때는 이슈 댓글로 미리 알림 / 풀 리퀘스트를 작게, 자주
- 각자 1 회 이상 충돌을 해결하고 회의록 활동 로그에 기록하는 것이 활동 규칙 항목입니다. 9 주차에 부록 D-2 실습으로 한 번씩 경험합니다

8.8 자주 하는 실수와 복구 (수업 12 주차 '실무 사례와 함께 Git 다루기' 기능 활용)

상황	복구 방법
방금 커밋에 파일을 빠뜨렸거나 메시지를 잘못 씀 (Push 전)	빠뜨린 파일을 스테이지에 올리고 커밋 옵션 → 마지막 커밋 정정(어멘드) → 커밋. 이미 Push한 커밋은 정정하지 말고 새 커밋으로
main에서 그냥 작업하고 커밋함 (Push 전)	소스트리 브랜치로 새 브랜치 feature/...를 만들면 그 커밋이 새 브랜치로 감 → main은 origin/main 커밋 우클릭 → 이 커밋까지 현재 브랜치 초기화(Hard)로 되돌림
Push가 거부됨 (rejected)	Pull → 충돌이 있으면 8.7절 → Push. 강제 푸시는 나 혼자 쓰는 브랜치에서만(수업 12 주차) , main에는 절대 금지
병합된 커밋·풀 리퀘스트를 되돌리고 싶음	공용 브랜치에서는 리셋 대신 리버트 : 소스트리에서 커밋 우클릭 → 커밋 되돌리기, 또는 GitHub 닫힌 풀 리퀘스트의 Revert 버튼 → 되돌리기 풀 리퀘스트 (수업 12·13주차)
급한 버그 때문에 다른 브랜치로 가야 하는데 작업 중인 파일이 있음	소스트리 스테시에 잠시 넣어 두고 브랜치 이동 → 돌아와서 스테시 적용(적용 후 삭제 체크) (수업 12주차)
다른 브랜치의 커밋 하나만 내 브랜치에 필요함	그 커밋 우클릭 → 체리 픽 (수업 12주차). 자주 쓸 일은 없음 — 보통은 풀 리퀘스트로 main에 합친 뒤 Pull
토큰·비밀번호·개인정보를 커밋함	파일을 지우고 다시 커밋해도 이력에 남음 → 해당 토큰을 즉시 재발급 (GitHub 토큰은 Settings → Developer settings에서 삭제). 앞으로는 9.5절
소스트리 그래프에 GitHub 병합 결과가 안 보임	패치(fetch)로 그래프 갱신 → main 체크아웃 → Pull (수업 9주차 실습 8~12단계)
git merge 뒤 검은 화면(vim)에 갇힘	Esc → :wq → Enter. 6.3절의 core.editor 설정을 하면 VS Code가 대신 열림

9. Python 프로젝트 기본

9.1 Python 버전

- 3.12 이상 아무 버전. 팀원끼리 같은 버전(예: 3.13)이면 더 좋음. README 에 '테스트한 버전'을 적어 둠
- 학기 중에는 설치한 버전을 바꾸지 않기 (10 월 1 일 출시 예정인 3.15 로 갈아타지 않기)

9.2 폴더 구조

```
moneylog/
├─ main.py           # 시작 파일: 메뉴를 보여 주고 각 기능을 호출
├─ add.py            # 기능 1 (팀원 A)      ← 기능마다 파일 1개, 담당자 1명
├─ summary.py       # 기능 2 (팀원 B)
├─ budget.py        # 기능 3 (팀원 C)
├─ chart.py         # 기능 4 (팀원 D)
├─ storage.py       # 공통: 파일 읽기·쓰기 (개발 리드)
├─ requirements.txt  # 사용하는 라이브러리 목록
├─ data/            # 샘플 데이터 (작은 파일만)
├─ docs/            # 회의록, 팀 약속, 스크린샷
├─ README.md LICENSE CONTRIBUTING.md CODE_OF_CONDUCT.md
└─ .gitignore
```

[왜?] 기능마다 파일을 나누고 담당자를 정하면 4명이 같은 파일을 고칠 일이 줄어 충돌이 거의 나지 않습니다. main.py와 공통 파일(storage.py)만 여럿이 손대므로, 이 파일을 고칠 때는 이슈 댓글로 먼저 알립니다.

9.3 설치와 실행 (README 에 그대로 적을 내용)

```
# 1) 저장소 받기 (소스트리 Clone 또는 아래 명령)
git clone https://github.com/<팀장아이디>/moneylog.git
cd moneylog

# 2) 라이브러리 설치 (requirements.txt에 적힌 것을 한 번에)
pip install -r requirements.txt

# 3) 실행
python main.py
```

- 새 라이브러리를 쓰면 requirements.txt 에 한 줄 추가(예: pandas)하고 같은 폴 리퀘스트에 포함. 팀원은 Pull 뒤 pip install -r requirements.txt 를 다시 실행
- 웹(Streamlit)이나 봇은 실행 명령이 다르므로 README '실행' 절에 정확한 명령을 적음 (예: streamlit run main.py)

9.4 코드 습관 (수업 3 주차 '스타일 가이드')

- Python 표준 스타일 PEP 8 을 따릅니다. VS Code Python 확장이 밑줄로 알려 주는 것만 고쳐도 충분
- 함수는 한 가지 일만, 30 줄 안팎. 파일 하나가 300 줄을 넘으면 나눔
- 이름은 영문으로 뜻이 보이게(monthly_total), 주석과 문자열은 한글 가능
- 파일을 읽고 쓸 때는 `open(path, encoding="utf-8")` — Windows 에서 한글 깨짐 방지
- 화면 출력(print, 입력)과 계산(함수)을 나누면 다른 팀원이 읽고 고치기 쉬움

9.5 저장소에 올리면 안 되는 것

올리지 말 것	대신
API 키, 봇 토큰, 비밀번호, 개인정보	config_example.py에 키 이름만 적어 올리고, 실제 값은 각자 PC의 config.py에 (.gitignore에 config.py 추가). 올라갔으면 즉시 재발급
큰 데이터 파일(수십 MB)	data/에는 5MB 이하 샘플만. 원본은 다운로드 링크를 README에
__pycache__/, 가상환경 폴더, 편집기 설정	저장소 생성 시 고른 Python .gitignore가 자동으로 제외함

10. 활동 규칙 (평가 30 점)

활동 규칙은 '얼마나 꾸준히, 규칙대로 협업했는가'를 GitHub 기록으로 확인하는 항목입니다. 10개 항목 × 3점. 각 항목은 수업에서 배운 주부터 적용하고, 중간고사 주간(10/19~10/25)은 최소 기준에서 뺍니다.

10.1 규칙 표

#	항목	활동 기준	적용 기간(최소 총량)	단위	확인 방법
1	커밋	각자 주 2회 이상	6주차(10/9)~12/13, 중간고사 주 제외 8주 → 16회	각자	저장소 Insights → Contributors
2	PR	각자 주 1회 이상 풀 리퀘스트 생성·병합	9주차(10/30)~12/13, 7주 → 7회	각자	Pull requests → 검색 is:pr author:@me
3	리뷰	풀 리퀘스트마다 다른 팀원 1명 이상이 리뷰(Approve)한 뒤 병합. 리뷰어는 돌아가며	9주차~	팀	각 풀 리퀘스트의 리뷰 기록
4	저장소 기여	개인(수업 3주차 실습 저장소 OSS), 팀 저장소, 타 팀 저장소(fork 커밋·이슈 댓글·받은 풀 리퀘스트 리뷰 중 1개) — 3개 이상	3~13주차	각자	프로필 활동
5	기여(외부 PR)	팀이 맡은 타 팀 이슈 1개를 풀 리퀘스트로 해결해 병합 (외부 공개 저장소는 선택)	11~13주차	팀	타 팀 저장소 Pull requests
6	Issue 작성	각자 주 1회 이상, 작업 전에 등록	6주차~12/13, 중간고사 주 제외 8주 → 8회	각자	Issues → 검색 is:issue author:@me
7	브랜치 전략	feature/기능이름 브랜치 사용, 9주차부터 main 직접 커밋 없음	7주차~	팀	브랜치 목록, 보호 규칙
8	Conflict 해결	각자 1회 이상 충돌 해결(9주차 실습 포함) + 회의록에 기록	9주차~	각자	회의록, 병합 커밋
9	문서화	README(소개·설치·실행), CONTRIBUTING, LICENSE, 행동 수칙	최종 기준	팀	저장소 첫 화면
10	회의록	수업 있는 주마다 1회, 총 10회 LMS 제출 (12장 날짜표)	5주차~12/11, 중간고사·최종발표 주 제외	팀	LMS

채점: 항목당 충족 3점 / 부분 충족(기준의 70% 이상) 1.5점 / 미충족 0점. '각자' 항목은 개인별로, '팀' 항목은 팀 전원 동일하게 반영됩니다.

[참고] 숫자는 최소 기준이고, 매주 조금씩 하면 자연스럽게 넘습니다. 기준을 채우려는 빈 커밋·의미 없는 이슈는 집계에서 뺍니다. 중간발표 전까지 커밋이 적더라도 7~9주차에 브랜치·풀 리퀘스트를 시작했다면 정상 진행입니다.

10.2 매주 스스로 확인하는 방법

확인할 것	방법
내 풀 리퀘스트	저장소 Pull requests 탭 검색창에 is:pr author:@me (병합된 것만: is:pr is:merged author:@me)

확인할 것	방법
내가 리뷰한 풀 리퀘스트	is:pr reviewed-by:@me
내 이슈	Issues 탭 검색창에 is:issue author:@me
내 커밋 수	저장소 Insights → Contributors (기간을 드래그로 조절)
팀 전체	Insights → Pulse (최근 1주 요약)
주간 기록	회의록 '활동 로그' 표에 각자 숫자 기입 → 팀장 확인

10.3 항목별 정의·예시·목적

항목	정의	구체적 예시	활동 목적
커밋	코드·문서 변경을 Git에 저장한 기록	소스트리 커밋 — 메시지 거래 입력 기능 추가 (#12)	변경 이력을 기록하고 추적 가능하게 만들기
PR	브랜치 작업을 main에 반영해 달라는 요청	feature/add-record → main 풀 리퀘스트, 설명에 Closes #12	협업자와 변경 사항 공유, 리뷰·병합 절차
리뷰	다른 팀원이 코드를 검토하고 판정	Files changed에서 [필수] 빈 입력 처리 필요 코멘트 후 Request changes	코드 품질 향상, 버그 사전 발견, 소통 훈련
저장소 기여	저장소에 커밋·풀 리퀘스트·이슈 등 실질 활동	개인 실습 저장소 커밋, 팀 저장소 개발, 타 팀 fork 커밋	실질적 활동으로 Contribution 그래프 생성
기여(외부 PR)	타 팀·외부 저장소에 풀 리퀘스트로 개선	타 팀 good first issue 해결 풀 리퀘스트	오픈소스 기여 경험, 낯선 코드 읽기
Issue 작성	할 일·버그·질문을 등록	[버그] 한글 입력 시 깨짐 + 재현 절차	작업 분담, 문제 추적, 진행 상황 가시화
브랜치 전략	기능별 브랜치로 작업 분리	feature/add-record, fix/csv-encoding	안전한 협업, 충돌 예방
Conflict 해결	같은 부분을 두 사람이 고쳐 생긴 충돌 정리	'This branch has conflicts' → 8.7절	실무에서 자주 겪는 상황 연습
문서화	외부인이 이해·실행·기여할 수 있게 정리	README, CONTRIBUTING, LICENSE, 행동 수칙	사용자·기여자를 위한 안내
회의록	주간 논의·결정·활동 로그 기록	docs/meetings/2026-10-01.md + PDF LMS 제출	의사결정 근거, 활동 기록의 근거

11. 타 팀 저장소에 기여하기 (11~13 주차)

11.1 개요와 매칭

- 8팀이 순환으로 짝을 이룹니다: 1팀 → 2팀, 2팀 → 3팀, ..., 8팀 → 1팀 (화살표 방향으로 기여).
- 각 팀은 상대 팀 저장소의 **이슈 1개**를 맡아 풀 리퀘스트 1개로 해결하고 병합까지 확인합니다. 동시에 우리 저장소도 다른 팀의 풀 리퀘스트 1개를 받아 리뷰·병합합니다
- 일정: 11주차(11/13~19) 이슈 선택·배정 요청·Fork → 12주차(~11/26) 풀 리퀘스트 제출 → 13주차(~12/3) 리뷰 반영·병합 확인
- 수업 11주차 '개발자 C'의 절차(포크 → 풀 리퀘스트 → 승인·병합)를 실제 팀 저장소로 해 보는 것입니다. 크기보다 절차가 중요합니다

11.2 받는 팀의 준비 (11주차 수업 전까지, 문서 담당)

1. **good first issue** 라벨 이슈 **2개 이상** 등록. 상대 팀이 그중 1개를 고릅니다. 각 이슈에 배경 / 할 일 / 완료 조건 / 관련 파일 이름을 적음 (예: 'README에 Windows 실행 예시 추가', 'summary.py에 빈 데이터 처리' --- json 출력 옵션)
2. CONTRIBUTING.md에 '타 팀 기여자' 절이 있는지 확인 (부록 C-2 템플릿에 포함)
3. 배정 요청 댓글이 오면 24시간 안에 답하고 Assignees에 지정. 받은 풀 리퀘스트는 48시간 안에 리뷰(Approve 또는 구체적인 수정 요청). 첫 기여자에게는 친절하게

11.3 기여하는 절차 (팀 단위, 수업 11주차 그대로)

1. 팀 회의에서 상대 팀의 **good first issue** 중 1개를 고르고, 이슈에 댓글: '○○팀이 이 이슈를 맡겠습니다. 11/26까지 풀 리퀘스트 올리겠습니다.' → 담당자 지정 확인
2. 팀장: 상대 팀 저장소 페이지 → 오른쪽 위 Fork → Create fork (팀장 계정에 복사본이 생김) → fork의 Settings → Collaborators에 팀원 3명 추가 (7.2와 같은 화면)
3. 역할 나누기: 구현 1~2명, 실행 확인 1명, 풀 리퀘스트 설명·회의록 기록 1명. 4명이 fork에 커밋하면 각자 '저장소 기여 3개'의 세 번째가 됨
4. 각자 fork를 소스트리 + (새 탭) → Clone — 폴더는 팀 저장소와 **다른 폴더**. feature/이슈이름 브랜치를 만들어 작업(8.1) → 커밋 → Push
5. 구현 담당: fork 페이지 → Contribute → Open pull request → base repository가 **상대 팀 원본**, head repository가 우리 fork인지 확인 → 설명(무엇/왜 Closes #번호/확인 방법/참여 팀원) → Create pull request
6. 리뷰 코멘트가 오면 24시간 안에 수정 → 같은 브랜치에 커밋·Push하면 풀 리퀘스트에 반영 → 병합되면 링크를 회의록 활동 로그에 기록

[TIP] 수업에서는 fork의 main에 바로 커밋했지만, 팀원 4명이 같은 fork에서 작업하므로 브랜치를 만드는 편이 안전합니다. 원본이 바뀌어 fork가 오래되면 fork 페이지의 Sync fork 버튼으로 최신화합니다.

기여 예절 (수업 7주차 '사람들이 중요하다', 9주차 '기여하기')	이유
상대 팀 CONTRIBUTING.md를 먼저 읽고 그대로 따름	우리 팀 규칙과 다를 수 있음. 규칙 준수가 곧 기여 능력
이슈 없이, 배정 없이 시작하지 않음	두 팀이 같은 일을 하는 사고 방지
이슈 범위 안에서만 고침. 구조 제안은 이슈로	상대 코드를 대규모로 뜯어고치지 않기
AI로 만든 코드는 직접 실행해 확인하고 설명에 표기	검증 없는 코드는 거절 사유
병합이 끝나면 감사 댓글, 받은 풀 리퀘스트에는 감사 댓글	오픈소스 커뮤니티의 기본 예절

11.4 외부 오픈소스 프로젝트 기여 (선택)

- 타 팀 기여 외에 실제 오픈소스에 기여하고 싶으면 수업 6 주차 '프로젝트 찾기'의 기준(목표·요건·후보 모으기, CONTRIBUTING 과 이슈 트래커가 정비된 프로젝트)으로 고릅니다
- 연습용: github.com/firstcontributions/first-contributions — 초보자용 공개 저장소. Contributors.md 에 자기 이름 한 줄을 추가하는 풀 리퀘스트를 보내면 보통 수 분~수 시간 안에 병합됩니다 (활동 집계에는 넣지 않지만 Fork 절차를 두려움 없이 익힐 수 있음)
- 2026 년 현재 많은 프로젝트가 AI 로 만든 저품질 풀 리퀘스트 때문에 오타·사소한 수정 풀 리퀘스트를 거절하고, 매년 10 월의 Hacktoberfest 도 풀 리퀘스트 개수 집계를 중단했습니다. 외부 기여는 '개수'가 아니라 '실제로 필요한 개선'일 때만 받아들여집니다
- 병합된 외부 풀 리퀘스트는 최종보고서에 링크로 기록. 응답이 수 주 걸릴 수 있으므로 활동 규칙 5 번(타 팀 기여)을 대체하지는 않습니다

12. 회의록 (총 10 회)

횟수	수업이 있는 주마다 1회, 5주차(10/2)부터 4주차 보강(12/11)까지 총 10회 (중간고사 주와 최종발표 주는 없음 — 최종발표 주는 최종보고서로 대체)
마감	수업일 전날(목요일) 18:00, LMS 과제 탭. PDF, 파일명 [WEEK5] 회의록_팀명.pdf (주차 번호는 수업 회차, 12/11 보강 수업은 [WEEK4])
회의	정기 회의 60분. 첫 5분은 3장 로드맵의 이번 주 확인, 마지막 10분은 다음 주 할 일·담당 확정.
작성	문서 담당이 회의 중 실시간 작성, 끝날 때 전원 확인
저장	저장소 docs/meetings/YYYY-MM-DD.md (풀 리퀘스트로 올리면 활동 실적)
PDF 만들기	GitHub에서 해당 .md 파일을 열고 브라우저 인쇄(Ctrl+P) → 'PDF로 저장'

12.1 제출 날짜

#	파일명	마감(목 18:00)	기간	비고
1	[WEEK5] 회의록_팀명	10/1	9/11~9/30	제안서와 함께
2	[WEEK6] 회의록_팀명	10/8	10/1~10/7	온라인 수업(10/9) 주
3	[WEEK7] 회의록_팀명	10/15	10/8~10/14	
—	(8주차 중간고사 10/23)	—	10/15~10/28	회의록 없음
4	[WEEK9] 회의록_팀명	10/29	10/15~10/28	2주차, 시험 뒤 재개
5	[WEEK10] 회의록_팀명	11/5	10/29~11/4	중간보고서와 함께
6	[WEEK11] 회의록_팀명	11/12	11/5~11/11	
7	[WEEK12] 회의록_팀명	11/19	11/12~11/18	
8	[WEEK13] 회의록_팀명	11/26	11/19~11/25	
9	[WEEK14] 회의록_팀명	12/3	11/26~12/2	
10	[WEEK4] 회의록_팀명	12/10	12/3~12/9	12/11 보강(4주차) 수업 전, 릴리스 확인 기록

12.2 회의록 내용 (부록 B 양식)

- 기본 정보: 주차, 일시, 방식, 참석·불참, 서기
- 지난주 결정 사항 점검
- 팀원별 진행 상황: 한 일(이슈·풀 리퀘스트 번호), 막힌 점
- 논의 내용과 결정 사항: 결정은 이유와 함께 한 줄씩
- 다음 주 할 일: 담당·마감·이슈 번호
- 활동 로그: 각자 이번 주 커밋·풀 리퀘스트·리뷰·이슈·충돌 해결 횟수 + 부록 A 점검표 체크

[TIP] 회의를 짧게 끝내는 형식: 각자 2분씩 '한 일 / 막힌 것 / 다음 할 일'을 말하는 체크인으로 시작하면 15분 안에 현황 파악이 끝납니다. 막힌 것은 회의에서 풀지 말고 담당자를 정해 회의 뒤 둘이 해결하세요.

13. 프로젝트 중간발표 (10 주차 11/6)

중간발표는 '지금까지 무엇이 동작하고, 남은 6주를 어떻게 쓸 것인가'를 보여 주는 자리입니다. 완성도보다 **실제로 실행되는 MVP(기능 2개)**와 **협업 기록**이 핵심입니다.

13.1 중간보고서

제출 마감	11/5(목) 18:00 LMS 과제 탭 (회의록 #5와 함께)
형식	PDF, 5~10쪽. 파일명 [중간보고서] 팀명.pdf
원칙	GitHub에서 확인할 수 있는 사실(링크·캡처)을 근거로. 계획과 달라진 것은 숨기지 않고 이유와 조정안을 씀

#	항목	들어갈 내용	분량
1	진행 요약	제안서 대비 진행률 표: 기능 4개 각각 계획 / 현재 상태(완료·진행·미착수) / 담당 / 관련 풀 리퀘스트 번호	1쪽
2	MVP 시연	동작하는 기능 2개의 실행 화면 캡처와 설명, README대로 설치·실행되는지 확인 결과	2~3쪽
3	협업 기록	Insights → Contributors·Pulse 캡처, 팀 통계(커밋·풀 리퀘스트·이슈·리뷰 수), 브랜치 그래프 캡처(소스트리), 충돌 해결 사례 1개	1~2쪽
4	타 팀 기여 진행	말은 상대 팀 이슈 링크 상태(배정/작업 중/풀 리퀘스트 제출), 우리 저장소에 온 요청 처리 상태	1쪽
5	문제와 대응	기술·협업 문제 각 1개 이상: 상황 → 시도 → 해결(또는 도움 필요)	1쪽
6	남은 계획	11~15주차 주차별 계획, 기능 동결(11/27)·릴리스(12/4~11) 일정, 기능 범위 조정이 있으면 그 내용	1쪽

13.2 중간발표

시간	팀당 발표 10분 + 질의 5분. 팀원 전원 1회 이상 발언
시연	MVP 기능 2개를 실제로 실행 (녹화 영상 백업 준비). 저장소 Insights 그래프를 화면에 띄워 협업 기록 설명
자료 구성(권장)	진행 요약 / MVP 데모 / 협업 기록 / 타 팀 기여 / 문제와 대응 / 남은 계획 — 6~8장
제출	발표 후 당일 이메일: shoh0320@pknu.ac.kr, 제목 [OSS 중간] 팀명, 발표자료(PPT/PDF) 첨부
평가	중간보고서 5점 + 발표 5점 — 기준은 15.2점

[TIP] 중간발표에서 받은 질문과 지적은 그 자리에서 이슈로 등록하세요. 최종보고서 '문제와 해결' 장의 재료가 되고, 활동 규칙의 이슈 실적도 됩니다.

14. 마무리: 릴리스·최종보고서·최종발표 (13~15 주차)

14.1 v1.0.0 태그와 릴리스 (14 주차, 수업 9 주차 '프로그램 출시하기')

버전은 수업에서 배운 대로 X.Y.Z(메이저.마이너.메인터너스)를 씁니다.

- 13주차(11/27)부터 기능 동결: 새 기능 풀 리퀘스트 금지, 버그 수정·문서·정리만. 리뷰·품질 담당 주도로 1.3 절 완성 기준 5개 점검 → 부족한 것은 이슈로 등록해 해결
- README 최종 확인(14.2) → 풀 리퀘스트로 병합
- 14주차(12/4): 소스트리에서 main 체크아웃 → Pull → 상단 태그 → 태그 이름 v1.0.0 → 태그 추가 → Push 창에서 모든 태그 푸시 체크 → Push (수업 9 주차 절차)
- GitHub 저장소 → Releases → Draft a new release → Choose a tag 에서 v1.0.0 선택 → 제목 v1.0.0 → 본문에 기능 4개 요약 + 설치·실행 방법 + 스크린샷 → Publish release (태그 페이지의 zip 다운로드만으로도 인정되지만 릴리스 노트가 있으면 결과물 점수에 유리)
- 4주차 보강 주(12/7~11): 팀원 1명이 새 폴더에 클론 → `pip install -r requirements.txt` → README 의 실행 명령 → 기능 4개 확인. 결과를 회의록 #10에 기록. 안 되면 README 를 고쳐 v1.0.1

[TIP] 발표 당일 네트워크·봇 장애에 대비해 **시연 녹화 영상**을 반드시 준비하세요. 스크린샷 3장(입력 → 처리 → 결과)을 README에 넣으면 데모 GIF를 대신할 수 있습니다.

14.2 README 필수 항목 7개 (수업 13 주차 '프로젝트에 꼭 포함할 파일')

#	항목	내용
1	제목·한 줄 소개	프로젝트 이름, 무엇을 하는 프로그램인지 한 문장
2	스크린샷	실행 화면 1장 이상
3	주요 기능	기능 4개 표 (이름·설명·담당)
4	설치·실행	<code>git clone</code> → <code>pip install -r requirements.txt</code> → <code>python main.py</code> (9.3), 테스트한 Python 버전
5	사용법	실제 명령·메뉴 예시 3개 이상과 결과 (수업 13주차 '사용자 가이드' 역할)
6	팀원·기여	이름·역할·GitHub 아이디 표, CONTRIBUTING·행동 수칙 링크
7	라이선스	MIT License — see LICENSE 한 줄

14.3 최종보고서

제출 마감	12/17(목) 18:00 LMS 과제 탭
형식	PDF, 표지·목차 포함 10~20쪽. 파일명 [최종보고서] 팀명.pdf
원칙	GitHub에서 확인할 수 있는 사실(링크·캡처)을 근거로. 계획과 달라진 부분은 숨기지 않고 이유를 씀. 중간보고서 내용을 갱신·확장하면 됨

#	장	들어갈 내용
1	프로젝트 개요	문제·목표·결과 요약, 저장소 주소, v1.0.0 태그·릴리스 주소
2	최종 기능	기능 4개 표(계획 vs 실제, 담당, 실행 화면)
3	프로그램 구조	파일·모듈 구성(9.2), 데이터 흐름, 사용 라이브러리
4	협업 기록	Insights 캡처(Contributors, Pulse), 팀 통계 표(커밋·풀 리퀘스트·이슈·리뷰 수), 브랜치 그래프 캡처(소스트리), 충돌 해결 사례 1~2개
5	개인별 기여	팀원별: 담당 기능, 대표 풀 리퀘스트 2개 링크, 리뷰 활동
6	타 팀 기여	맡은 이슈·풀 리퀘스트 링크(병합 여부), 우리 저장소에 병합한 타 팀 풀 리퀘스트 링크, 배운 점
7	문제와 해결	기술·협업 각 1개 이상: 상황 → 시도 → 해결 → 배운 점 (중간발표 지적 사항 반영 포함)
8	회고와 향후 계획	잘한 점 / 아쉬운 점 / 다음에 시도할 것, 릴리스 이후 계획
+	부록	회의록 10회 링크, 부록 A 점검표 최종본, 팀 약속

14.4 최종발표 (15 주차 12/18)

시간	팀당 발표 15분(시연 포함) + 질의 5분. 팀원 전원 1회 이상 발언
시연	실제 실행 (녹화 영상 백업 필수). 발표자료에 GitHub 협업 기록(Insights 그래프, 풀 리퀘스트 목록) 포함
자료 구성(권장)	문제·목표 / 데모 / 기능 4개 / 구조 / 협업 기록 / 개인 기여 / 타 팀 기여 / 문제와 해결 / 회고
준비	4주차 보강(12/11) 수업 시간이 팀별 준비 시간: 발표 리허설, 시연 녹화, 질의 예상 답변
제출	발표 후 당일 이메일: shoh0320@pknu.ac.kr, 제목 [OSS 최종] 팀명, 발표자료(PPT/PDF) 첨부
질의 예시	'이 부분은 왜 이렇게 나왔나요?', '충돌은 어떻게 해결했나요?', '타 팀 풀 리퀘스트에서 배운 점은?'
평가	최종보고서 10점 + 발표 5점 — 기준은 15.3점

15. 평가

프로젝트 점수 100점은 과목 성적의 50%로 환산됩니다 (출결 10%, 중간고사 40%, 팀 프로젝트 50%).

구분	내용	세부	점수
보고서 및 발표 (40)	프로젝트 제안서	제안서 (15.1절)	10
		발표	5
	중간보고서	중간보고서 (15.2절)	5
		중간발표	5
	최종보고서	최종보고서 (15.3절)	10
		최종발표	5
프로젝트 활동 (60)	활동 규칙 준수 여부	10개 항목 × 3점 (10장)	30
	결과물	완성 기준 5개 (15.5절)	30

'각자' 활동 항목(커밋·PR·저장소 기여·이슈·충돌 해결)은 개인별 GitHub 기록으로 채점되므로 같은 팀이라도 점수가 다를 수 있습니다. 나머지는 팀 전원 동일합니다.

15.1 제안서 10 점 + 발표 5 점

항목	배점	만점 기준
문제 정의·목표	2	어떤 문제를 푸는지 구체적
기능 설계	3	기능 4개가 1인 1기능으로 나뉘고, 각 기능의 완료 조건이 확인 가능(4.4 표)
일정·역할	2	주차별 계획이 MVP(10주차)·동결(13주차)·릴리스(14주차)와 맞고, 역할 4개가 명확
협업 계획	2	브랜치·커밋·풀 리퀘스트·리뷰·이슈 규칙, 라이선스·행동 수칙
문서 완성도	1	7항목 형식 준수, 오타자 없음, 저장소 주소 기재
발표	5	10분 안에 7항목(2), 전원 발언·전달(2), 질문에 근거 있는 답변(1)

15.2 중간보고서 5 점 + 중간발표 5 점

항목	배점	만점 기준
진행 요약·계획 조정	1	기능 4개의 상태가 풀 리퀘스트 번호로 확인되고, 남은 계획이 현실적
MVP	2	기능 2개가 실제 데이터로 동작(캡처와 시연 일치)
협업 기록	1	Insights 캡처·통계 표·브랜치 그래프, 충돌 사례
타 팀 기여·문제 대응	1	말은 이슈 상태 기재, 문제 → 시도 → 해결 구조
발표	5	시연 성공(2), 구성·전달·전원 발언(2), 질의응답(1)

15.3 최종보고서 10 점 + 최종발표 5 점

항목	배점	만점 기준
기능·구조 설명	3	2~3장이 실제 저장소와 일치하고 그림·표로 설명
협업 기록	3	4장에 Insights 캡처와 수치 표, 충돌 사례
개인별·타 팀 기여	2	5~6장의 풀 리퀘스트·리뷰·타 팀 기여 링크가 확인 가능
문제 해결·회고	1	7~8장이 구체적(상황 → 시도 → 해결 → 배움)
형식	1	분량·파일명·목차·오탈자
발표	5	시연 성공(2), 구성·전달·전원 발언(2), 질의응답(1)

15.4 활동 규칙 30 점

10장의 규칙 표 10개 항목 × 3점. 항목당 충족 3점 / 부분 충족(70% 이상) 1.5점 / 미충족 0점.

15.5 결과물 30 점 (완성 기준 5 개)

항목	배점	만점 기준	부분 점수 예
기능 동작(기준 2)	12	기능 4개 모두 동작, 시연 성공	3개 9점, 2개 6점, 1개 3점
실행 재현(기준 1)	6	새 폴더에서 README대로 실행 성공, v1.0.0 태그	README 보완 후 성공 4점, 태그 없음 -2
문서(기준 3)	6	README 7항목, CONTRIBUTING·행동 수칙·LICENSE	항목 누락 1개당 -1
협업 절차(기준 4·5)	6	모든 기능이 풀 리퀘스트·리뷰로 병합, 타 팀 이슈 해결·병합, 받은 풀 리퀘스트 병합	main 직접 커밋 있음 -2, 타 팀 기여 미병합 -2, 받은 풀 리퀘스트 미처리 -2

15.6 감점·무효

행위	처리
다른 저장소 코드를 출처 표기 없이 복사(라이선스 위반 포함)	해당 기능 결과물 점수 0
AI로 만든 코드를 설명하지 못함	해당 풀 리퀘스트 활동 집계 제외
main 강제 푸시, 보호 규칙 해제, 리뷰 없는 병합	활동 규칙 7번 0점
허위 회의록·활동 로그	회의록 항목 0점
제출 마감(목 18:00) 초과	지각 제출은 각 제출물 총점에서 10% 감점

16. 도움 받기

16.1 막혔을 때 순서

1. 부록 E FAQ 에서 오류 메시지로 검색
2. 팀 채널에 질문 — 팀원 중 누군가는 같은 문제를 겪었을 확률이 높음. 24 시간 내 응답 규칙
3. 저장소 이슈에 `question` 라벨로 등록 (다른 팀도 볼 수 있음)
4. 교수 이메일(shoh0320@pknu.ac.kr) 또는 수업 시간 질문. 아래 양식으로 보내면 답이 빠름 (수업 3 주차 '질문하는 방법')

16.2 질문 양식

[팀명 / 이름] 하려던 일: 소스트리에서 `feature/add-record` 브랜치를 Push
 실행한 것: 상단 Push → 브랜치 체크 → Push 버튼
 오류 메시지(전체 텍스트): `remote: Permission to ... denied to ...`
 환경: Windows 11 / Git 2.51 / 소스트리 3.x / 저장소 주소
 이미 해 본 것: 소스트리 옵션 → 인증에서 계정 재로그인

16.3 AI 도구 (Copilot, ChatGPT, Claude 등)

- 사용해도 됩니다. 단, 풀 리퀘스트에 올린 코드는 **작성자가 설명할 수 있어야** 합니다. 발표·리뷰에서 '이 줄은 왜 필요한가?'에 답하지 못하면 집계에서 제외됩니다
- 좋은 사용: 오류 메시지 해석, 명령어 의미 질문, 내 코드의 문제점 찾기. 나쁜 사용: '프로젝트 통째로 만들어 줘', 실행해 보지 않은 코드 커밋, 토큰·개인정보 입력
- 학생 인증(GitHub Student Developer Pack)이 되면 GitHub Copilot 을 '무료로 쓸 수 있습니다 (선택)

16.4 참고 자료

주제	자료	주소
수업 자료	주차별 강의자료 PDF (3·6·7·9·11·12·13·14주차 가 Git·GitHub 실습)	Smart-LMS
Git 기초	Pro Git 한국어판 (2~3장)	git-scm.com/book/ko/v2
GitHub	GitHub Docs 한국어, GitHub Skills(실습 과정)	docs.github.com/ko , skills.github.com
소스트리	Atlassian 소스트리 지원 문서	support.atlassian.com/sourcetree
Python	공식 튜토리얼 한국어	docs.python.org/ko/3/tutorial
오픈소스 기여	Open Source Guides 한국어, first-contributions	opensource.guide/ko , github.com/firstcontributions/first-contributions
라이선스	Choose a License	choosealicense.com
공공데이터	공공데이터포털, 부산 공공데이터포털	data.go.kr , data.busan.go.kr

부록 A. 주차별 개인 점검표

매주 회의록에 해당 주차를 복사해 각자 체크합니다. 항목은 '/'로 구분되어 있고 체크 칸은 항목 순서대로입니다.

주차	체크 항목	완료
2주 (9/11)	팀 구성·역할·팀 약속 / GitHub 계정 / 팀 저장소 초대 수락 / 주제 후보 3개	☐ ☐ ☐ ☐
3주 (9/18)	토큰 발급, Git 설치·설정 / 수업 실습 저장소(OSS) 커밋·푸시 / 주제 확정·담당 기능 결정	☐ ☐ ☐
4주 휴강 (9/25)	제안서 완성·제출(10/1 18:00) / 회의록 #1(10/1)	☐ ☐
5주 (10/2)	제안서 발표(전원 발연) / (팀장) 기본 파일 6종 업로드 / 담당 기능 이슈 3개 이상 등록 / 회의록 #2(10/8)	☐ ☐ ☐ ☐
6주 (10/9 온 라인)	온라인 수업 수강 / 소스트리 설치·GitHub 연결 / 팀 저장소 클론 / 자기소개 파일 커밋·Push / 회의록 #3(10/15)	☐ ☐ ☐ ☐ ☐
7주 (10/16)	feature/기능이름 브랜치 생성 / 첫 동작 코드 커밋 / 브랜치 Push	☐ ☐ ☐
8주 (10/23)	중간고사 — 회의록·최소 기준 면제 / 시험 뒤 브랜치 작업 재개 / 회의록 #4(10/29)	☐ ☐ ☐
9주 (10/30)	첫 풀 리퀘스트 → 리뷰 → 병합 / 팀원 풀 리퀘스트 리뷰 1회 / 충돌 해결 실습(부록 D-2) / (팀장) 브랜치 보호 / MVP 기능 2개 병합 / 회의록 #5(11/5)	☐ ☐ ☐ ☐ ☐ ☐
10주 (11/6)	중간보고서(11/5 18:00) / 중간발표·MVP 시연 / (문서 담당) good first issue 2개 이상 등록 / 회의록 #6(11/12)	☐ ☐ ☐ ☐
11주 (11/13)	상대 팀 이슈 선택·배정 요청 / (팀장) Fork·협력자 등록 / fork에서 작업 / 우리 저장소에 온 요청 응답 / 회의록 #7(11/19)	☐ ☐ ☐ ☐ ☐
12주 (11/20)	상대 팀 풀 리퀘스트 제출(~11/26) / 받은 풀 리퀘스트 리뷰 / 기능 3~4 구현 / 회의록 #8(11/26)	☐ ☐ ☐ ☐
13주 (11/27)	상대 팀 풀 리퀘스트 병합 확인(~12/3) / 받은 풀 리퀘스트 병합 / 기능 동결 / README·CONTRIBUTING·행동 수칙·LICENSE 정비 / 회의록 #9(12/3)	☐ ☐ ☐ ☐ ☐
14주 (12/4)	완성 기준 5개 점검 / v1.0.0 태그·릴리스 / 회의록 #10(12/10)	☐ ☐ ☐
4주차 보강 (12/11)	새 폴더 클론·실행 확인 / 최종보고서 초안 / 발표자료·시연 녹화	☐ ☐ ☐
15주 (12/18)	최종보고서(12/17 18:00) / 최종발표·시연(GitHub 협업 기록 포함) / 발표자료 이메일	☐ ☐ ☐

부록 B. 회의록 양식

docs/meetings/YYYY-MM-DD.md로 저장하고 PDF로 변환해 LMS에 제출합니다(12장). 아래를 양식을 사용하여 내용을 채웁니다.

[WEEK#] 회의록 - 팀명

- 일시: 2026-10-13 (화) 20:00 ~ 21:00
- 방식: 온라인(디스코드) / 대면(도서관 스터디룸)
- 참석: 홍길동, 김철수, 이영희, 박민수 / 불참: 없음
- 서기: 박민수

1. 지난주 결정 사항 점검

결정 사항	담당	완료	비고
저장 형식 CSV 확정	김철수	0	storage.py 올림
샘플 데이터 준비	이영희	x	이번 주

2. 팀원별 진행 상황

팀원	한 일 (이슈/풀 리퀘스트 번호)	막힌 점
홍길동	#3 입력 검증 이슈 등록	날짜 형식 검사 방법
김철수	#8 storage.py 커밋	없음

3. 논의 내용과 결정 사항 (이유 포함)

- 예산 저장은 별도 파일(budget.json)로 - 거래 CSV와 섞으면 통계가 복잡해져서

4. 다음 주 할 일

할 일	담당	마감	이슈
add 기능 첫 동작	홍길동	10/21	#3

5. 활동 로그 (이번 주)

팀원	커밋	풀 리퀘스트(생성/병합)	리뷰	이슈	충돌 해결	특이사항
홍길동	3	0/0	0	2	0	-
김철수	4	0/0	0	1	0	-

6. 이번 주 점검표 (부록 A 6주차)

- [x] 소스트리 설치 - [x] 클론 - [x] 자기소개 파일 커밋·Push - [x] 회의록 #3

부록 C. 템플릿 파일

7.3절에서 GitHub 웹의 `Add file` → `Create new file`로 넣는 파일입니다. `moneylog`, `<팀장아이디>` 등은 여러분의 값으로 바꿉니다.

C-1. README.md

```
# moneylog

터미널에서 쓰는 가계부 - 거래 입력, 월별 통계, 예산 경고, 그래프 저장

## 스크린샷
![실행 화면](docs/images/demo.png)

## 주요 기능
| 기능 | 설명 | 담당 |
|-----|-----|-----|
| add | 거래 입력·검증 | 홍길동 |
| summary | 월별·분류별 통계 | 김철수 |
| budget | 예산 설정·초과 경고 | 이영희 |
| chart | 그래프 PNG 저장 | 박민수 |

## 설치와 실행 (Python 3.12 이상, 3.13에서 테스트)
...
git clone https://github.com/<팀장아이디>/moneylog.git
cd moneylog
pip install -r requirements.txt
python main.py
...

## 사용법
1. 메뉴에서 `1`을 누르면 거래 입력 - 예: `2026-10-13, 12000, 식비, 점심`
2. `2`를 누르면 이번 달 합계와 분류별 합계 출력
3. `4`를 누르면 `docs/images/` 폴더에 그래프 PNG 저장

## 팀원
| 이름 | 역할 | GitHub |
|-----|-----|-----|
| 홍길동 | 팀장 | @hong-id |

기여 방법은 [CONTRIBUTING.md](CONTRIBUTING.md), 행동 수칙은 [CODE_OF_CONDUCT.md](CODE_OF_CONDUCT.md)를 참고하세요.

## 라이선스
MIT License - see [LICENSE](LICENSE)
```

C-2. CONTRIBUTING.md

기여 가이드

준비

1. Python 3.12 이상 설치 → 저장소 클론 → ``pip install -r requirements.txt``
2. 실행: ``python main.py``

작업 흐름 (팀원)

1. 작업 전에 이슈를 먼저 만들거나 기존 이슈에 댓글로 배정 요청
2. ``main``에서 ``feature/기능이름`` 브랜치를 만들어 작업 (`main`에 직접 커밋하지 않음)
3. 커밋 메시지: ``기능 설명 (#이슈번호)`` - 예: ``거래 입력 기능 추가 (#12)``
4. 올리기 전에 프로그램을 실행해 확인
5. 풀 리퀘스트 설명에 무엇을/왜(``Closes #이슈``)/어떻게 확인했나 3줄, 리뷰어 1명 지정
6. 승인 1개 뒤 작성자가 ``Merge pull request``

타 팀·외부 기여자

1. ``good first issue`` 라벨 이슈에 댓글로 배정을 요청하고, 담당자로 지정된 뒤 시작
2. 이 저장소를 ``Fork`` → 소스트리로 클론 → 브랜치를 만들어 작업
3. 수정 → 커밋 → Push → fork 페이지의 ``Contribute`` → ``Open pull request`` (base는 이 저장소의 ``main``)
4. 리뷰·질문 응답은 48시간 이내

행동 수칙

- 모든 참여자는 `CODE_OF_CONDUCT.md` 를 따릅니다

코드 규칙

- PEP 8 스타일, 기능마다 파일 1개, 파일 입출력은 ``encoding="utf-8"``
- 새 라이브러리는 `requirements.txt` 에 추가
- 토큰·비밀번호·개인정보는 절대 올리지 않음

AI 도구

- 사용 가능. 직접 실행해 확인한 코드만 올리고, 풀 리퀘스트 설명에 사용 여부를 적음

C-3. CODE_OF_CONDUCT.md (행동 수칙)

수업 3·13주차에서 '프로젝트에 꼭 포함할 파일'로 배운 행동 수칙입니다. 아래 짧은 버전을 참고해서 써도 되고, 표준 문서인 Contributor Covenant(contributor-covenant.org, 한국어 번역 있음)를 가져와도 됩니다.

행동 수칙

이 프로젝트의 모든 참여자(팀원, 타 팀 기여자, 사용자)는 서로를 존중하며 다음을 지킵니다.

환영하는 행동

- 리뷰·이슈·채팅에서 사람이 아닌 코드와 문제에 대해 말합니다.
- 질문을 환영하고, 처음 기여하는 사람에게는 절차를 구체적으로 안내합니다.
- 의견이 다를 때는 근거(실행 결과, 문서)를 들어 설명합니다.
- 요청·리뷰에는 48시간 안에 응답하고, 늦어지면 미리 알립니다.

허용하지 않는 행동

- 인신공격, 조롱, 차별적 표현, 비하
- 다른 사람의 작업을 동의 없이 삭제하거나 덮어쓰는 행위(강제 푸시)
- 개인정보·비밀 키를 공개 저장소에 올리는 행위

신고와 처리

- 문제가 있으면 팀장에게 알리거나 교수(shoh0320@pknu.ac.kr)에게 이메일로 알립니다.
- 반복되는 위반은 회의록에 기록하고 평가에 반영됩니다.

C-4. requirements.txt

사용하는 라이브러리를 한 줄에 하나씩. 표준 라이브러리(csv, json 등)는 적지 않음.
matplotlib

C-5. main.py (시작 파일 예시)

```
"""moneylog - 터미널 가계부. 메뉴를 보여 주고 각 기능을 호출한다."""

def main():
    while True:
        print("\n=== moneylog ===")
        print("1. 거래 입력   2. 월별 통계   3. 예산 관리   4. 그래프 저장   0. 종료")
        choice = input("선택: ").strip()
        if choice == "0":
            break
        elif choice == "1":
            print("준비 중 (담당: 홍길동, #3)") # 기능이 완성되면 add.run() 으로 교체
        elif choice == "2":
            print("준비 중 (담당: 김철수, #4)")
        elif choice == "3":
            print("준비 중 (담당: 이영희, #5)")
        elif choice == "4":
            print("준비 중 (담당: 박민수, #6)")
        else:
            print("0~4 중에서 선택하세요.")

if __name__ == "__main__":
    main()
```

C-6. .gitignore 에 추가할 줄 (저장소 생성 시 고른 Python 템플릿 아래에)

```
# ---- 프로젝트 추가 항목 ----
config.py          # 토큰.키를 넣는 파일 (config_example.py 만 올림)
data/*.csv
!data/sample.csv
.DS_Store
Thumbs.db
```

C-7. 풀 리퀘스트 설명 양식 (선택: .github/PULL_REQUEST_TEMPLATE.md 로 저장하면 자동 입력)

```
## 무엇을 바꿨나요?
-

## 왜 바꿨나요? (관련 이슈)
Closes #

## 어떻게 확인했나요?
- 실행 명령:
- 확인한 내용:

## 스크린샷 (화면이 바뀐 경우)

## AI 도구 사용: 없음 / 있음(도구, 범위)
```

부록 D. 소스트리·명령어 대응표와 충돌 실습

D-1. 소스트리 조작 ↔ Git Bash 명령 (수업 3·12·14 주차)

같은 일을 두 가지 방법으로 할 수 있습니다. 활동 집계에는 차이가 없습니다.

하려는 일	소스트리	Git Bash 명령
저장소 받기	Clone → 주소 → 폴더 → 클론	git clone <주소>
상태 보기	파일 상태, History	git status, git log --oneline --graph --all
최신화	main 체크아웃 → Pull	git switch main → git pull
브랜치 만들기·이동	브랜치 → 이름 → 새 브랜치 체크아웃 / 사이드바 더블클릭	git branch feature/x → git switch feature/x (또는 git switch -c feature/x)
커밋	커밋 → +로 스테이지 → 메시지 → 커밋	git add <파일> (전체는 git add .) → git commit -m "메시지"
마지막 커밋 정정	커밋 옵션 → 마지막 커밋 정정	git commit --amend (Push 전에만)
푸시	Push → 브랜치 체크 → Push	첫 푸시 git push -u origin feature/x, 이후 git push
원격 변화 확인	패치	git fetch
병합	체크아웃 → 대상 커밋 우클릭 → 병합	git switch main → git merge feature/x
충돌 해결 후	파일 스테이지 → 커밋 → Push	git add <파일> → git commit → git push
되돌리기(내 브랜치)	커밋 우클릭 → 이 커밋까지 현재 브랜치 초기화	git reset --hard HEAD~1 (Push 전에만)
되돌리기(공용 브랜치)	커밋 우클릭 → 커밋 되돌리기	git revert <커밋ID>
잠시 치우기	스테시 → 설명 입력 / 스테시 적용	git stash / git stash pop
태그	태그 → 이름 → 태그 추가 → Push(모든 태그 푸시)	git tag v1.0.0 → git push origin v1.0.0
브랜치 삭제	브랜치 우클릭 → 삭제	git branch -d feature/x

[참고] 수업 14주차에서 배우는 rebase와 reset --hard, 12주차의 강제 푸시는 이력을 바꾸는 기능이라 여럿이 쓰는 브랜치에서는 위험합니다. 이 프로젝트에서는 병합(merge)과 풀 리퀘스트, 되돌릴 때는 리버트만 사용하세요.

D-2. 충돌 해결 실습 (9 주차, 수업 9 주차 직후, 2인 1조 20분)

수업 9주차 실습(학생A·학생B가 같은 파일을 고쳐 충돌)과 같은 상황을 팀 저장소에서 일부러 만듭니다. A·B가 끝나면 C·D가 반복합니다.

1. 준비(팀장): docs/drill.md 에 현재 당번: (이름) 한 줄을 넣어 풀 리퀘스트로 병합
2. A와 B 모두: 소스트리 main 체크아웃 → Pull

3. A: feature/drill-a 브랜치 → 그 줄을 현재 당번: A 이름으로 수정 → 커밋 → Push → 풀 리퀘스트 → B 가 Approve → A 가 병합
4. B: (A 가 병합한 뒤, main 을 Pull 하지 말고) feature/drill-b 브랜치 → 같은 줄을 현재 당번: B 이름으로 수정 → 커밋 → Push → 풀 리퀘스트 → 'This branch has conflicts' 확인
5. B: 8.7 절 순서대로 해결(main Pull → 내 브랜치에 main 병합 → VS Code 에서 정리 → 커밋 → Push) → 충돌 표시가 사라지면 A 가 Approve → 병합
6. 회의록 활동 로그의 '충돌 해결' 칸에 B 1 회 기록. 이어서 C·D 반복(역할을 바꾸면 4 명 모두 1 회)

부록 E. 문제 해결 FAQ

오류 메시지의 핵심 단어로 이 표를 검색하세요(Ctrl+F). 해결되지 않으면 16.2절 양식으로 질문합니다.

증상	원인	해결
소스트리 Push가 거부됨 (rejected)	다른 팀원이 먼저 올림	Pull → (충돌 있으면 8.7절) → 다시 Push. 강제 푸시 금지
Push할 때 'Permission denied' / 인증 오류	로그인 계정이 다르거나 토큰 만료·협력자 미등록	소스트리 도구 → 옵션 → 인증에서 GitHub 계정 확인·재로그인(수업 6·11주차). 협력자 초대 수락했는지 확인
main에 직접 커밋해서 Push가 막힘	보호 규칙이 켜져 있음(9주차 이후)	정상 동작. 8.8절 둘째 줄 방법으로 커밋을 새 브랜치로 옮긴 뒤 풀 리퀘스트
풀 리퀘스트에 'This branch has conflicts that must be resolved'	같은 부분을 두 사람이 수정	8.7절
병합 버튼이 회색 — 'Review required'	승인 1개가 없음	리뷰어에게 Approve 요청. 본인은 본인 풀 리퀘스트를 승인할 수 없음
소스트리 그래프에 GitHub에서 병합한 결과가 안 보임	그래프 미갱신	패치 → main 체크아웃 → Pull (수업 9주차 실습 8~12단계)
ModuleNotFoundError: No module named 'pandas'	라이브러리 미설치	pip install -r requirements.txt. 여전히 안 되면 VS Code 오른쪽 아래 Python 버전이 설치한 것과 같은지 확인
python을 입력하면 Microsoft Store가 열림 / 'python은 내부 또는 외부 명령이 아닙니다'	설치 시 PATH 체크 누락	Python 재설치하며 Add python.exe to PATH 체크, 또는 py main.py로 실행
터미널에서 한글이 ?로 깨짐	인코딩	CMD/PowerShell에서 chcp 65001 후 재시도. 파일 입출력에는 encoding="utf-8"
Windows에서 warning: LF will be replaced by CRLF	출바꿈 안내	경고일 뿐 오류 아님. 무시해도 됨
Git Bash에서 git merge 뒤 검은 화면(vim)에 갇힘	병합 메시지 편집기	Esc → :wq → Enter. 6.3절 core.editor 설정으로 예방
병합했는데 이슈가 안 닫힘	Closes #n이 설명이 아닌 댓글에 있거나 오타	풀 리퀘스트 설명(첫 글)에 Closes #12. 이미 병합됐으면 이슈에서 수동 Close
Fork한 저장소가 원본보다 뒤처짐	원본에 새 커밋	fork 페이지의 Sync fork 버튼
협력자 초대를 받았는데 Push가 안 됨	초대 미수락(7일 만료)	GitHub 알림·이메일에서 Accept. 만료됐으면 팀장이 재초대
내 커밋이 GitHub 프로필 잔디(Contribution)에 안 뜸	이메일 불일치 또는 main에 아직 병합 안 됨	Git·소스트리의 이메일이 GitHub 계정 이메일과 같은지 확인(6.2). main에 병합된 커밋만 집계 (fork의 커밋은 원본에 병합되면 집계)
실수로 토큰·비밀번호를 커밋함	비밀 정보 유출	파일을 지워도 이력에 남음 → 해당 토큰 즉시 재발급 (GitHub Settings → Developer settings에서 삭제). 9.5절
작업 중인 파일이 있는데 급히 다른 브랜치로 가야 함	커밋하기엔 미완성	소스트리 스테시 → 브랜치 이동 → 돌아와서 스테시 적용 (수업 12주차)
팀원이 갑자기 연락 두절	개인 사정	2.4절: 팀장 연락 → 회의록 기록 → 2주 시 교수 이메일. 담당 기능은 팀이 임시 인수

부록 F. 팀 약속 양식

docs/team-agreement.md로 저장하고 2주차 회의에서 4명이 확인합니다.

팀 약속 – 팀명

작성일: 2026-09-11 / 확인: 홍길동, 김철수, 이영희, 박민수

1. 팀원과 역할

이름	역할	GitHub	담당 기능
홍길동	팀장	@hong-id	add
김철수	개발 리드	@kim-id	summary + 공통 코드
이영희	문서 담당	@lee-id	budget
박민수	리뷰·품질 담당	@park-id	chart

2. 소통

- 채널: 디스코드 서버 '팀명'
- 정기 회의: 매주 화요일 20:00~21:00, 온라인. 불참 시 전날까지 알림
- 응답: 채팅 24시간, 풀 리퀘스트 리뷰 48시간

3. 작업 규칙

- 작업 전 이슈 등록, 하루 작업 끝에 Push
- 브랜치·커밋·풀 리퀘스트 규칙은 안내서 8장
- main.py·공통 파일을 고칠 때는 이슈 댓글로 먼저 알림

4. 의사결정

- 회의에서 다수결, 동률이면 개발 리드 결정. 결정과 이유는 회의록에 기록

5. 문제 대응

- 1주 무응답 → 팀장 연락 / 2주 → 회의록 기록 + 교수 상담